

02.05.2026

Конкурентность. Многопоточность.

Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2024-2025

N * Новосибирский
государственный
университет
***НАСТОЯЩАЯ НАУКА**

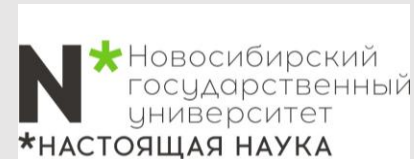


Давайте познакомимся



Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.



Адженда

Конкурентность

30 минут

**Синхронизация
поток**

30 минут

Синхронизация

30 минут

Процессы

30 минут

**Многопоточное
выполнение**

30 минут

**Синхронизация
процессов**

30 минут

Адженда

Конкурентность

30 минут

**Синхронизация
потоков**

30 минут

Синхронизация

30 минут

Процессы

30 минут

**Многопоточное
выполнение**

30 минут

**Синхронизация
процессов**

30 минут

Введение в конкурентность

Конкурентность означает, что несколько участков логики программы выполняются одновременно. Многие современные программные системы конкурентны, поскольку программам необходимо совершать сразу несколько разных действий.

Можно сказать, это мощный инструмент, позволяющий писать код, который выполняет несколько заданий одновременно, и его поддержка обычно находится в **сердце операционной системы — в ядре**.

Примеры:

- вы можете перемещаться по вебстраницам, загружая при этом какие-то файлы
- потоковая загрузка видео, когда вы смотрите что-то на YouTube
- Microsoft Word - механизмы проверки правописания и форматирования, выполняющиеся в параллель

Одновременное выполнение нескольких программ звучит чудесно, но, как часто бывает с технологиями, конкурентность несет с собой не только преимущества, но и определенные проблемы. Действительно, мало какая технология в истории компьютерных наук вызывала столько головной боли!!!

Конкурентная и параллельная работа — разные вещи.

Базовые концепции, относящиеся к конкурентности: задания, планирование, чередование, состояние и разделяемое состояние. Вдобавок стоит отметить: большинство из этих концепций являются абстрактными и применимы к любой конкурентной системе, а не только к C.



Параллелизм

Параллелизм означает, что два задания выполняются одновременно, или в параллель.

Потому что **параллельность** подразумевает протекание двух событий в один и тот же момент. В случае с **конкурентной** системой это не так; прежде чем выполнять другое задание, она должна остановить текущее.

Для параллельного выполнения необходимо разделить и изолировать вычислительные блоки, каждый из которых назначается определенному заданию. Например, в компьютерной системе таким блоком выступает ядро процессора, которое может выполнять по одной инструкции за раз.

Для параллельного выполнения двух задач компьютерной системе нужно по меньшей мере два отдельных и независимых вычислительных блока. Современные процессоры содержат внутри несколько ядер, которые являются этими самыми блоками.

Параллелизм возможен только в задачах, которые можно распараллелить. Это означает, что алгоритм можно разделить на части и запустить на разных вычислительных блоках. Однако на сегодня большинство алгоритмов, которые мы пишем, по своей природе последовательны, а не параллельны.

Имея многоядерный процессор, вы все равно должны назначить каждый поток выполнения определенному ядру. И если одно ядро получит несколько потоков, то их нельзя будет выполнить параллельно. В результате вы сможете наблюдать конкурентное поведение.



Конкурентность

Конкурентность зиждется на идее многозадачности. Если ваша система выполняет сразу несколько задач, то это вовсе не означает их параллельную работу. В данном процессе может участвовать планировщик заданий; он просто очень быстро переключается между разными заданиями, выполняя некую крошечную часть каждого из них за довольно короткий промежуток времени.

Это, несомненно, происходит на компьютере с одним вычислительным блоком.

Если планировщик достаточно быстрый и беспристрастный, то вы не заметите переключения между заданиями; вам будет казаться, что они работают параллельно. В этом вся магия конкурентности, и именно поэтому она применяется в большинстве широко известных операционных систем, включая Linux, macOS и Microsoft Windows.

Конкурентность можно считать имитацией параллельного выполнения заданий на одном вычислительном блоке.

Multics была одной из первых операционных систем с поддержкой многозадачности и управления одновременными процессами. Почти все ОС могут выполнять конкурентные задания, используя многозадачность. В POSIX эта возможность явно обозначена в стандарте.



Планировщик заданий

У всех многозадачных операционных систем в ядре должен быть **планировщик заданий**, или просто планировщик. Ниже перечислены **несколько фактов о планировщике**:

- Есть очередь заданий, ждущих, когда их выполнят. Задание — просто наборы операций, которые должны быть выполнены в отдельных потоках.
- Очередь обычно поддерживает приоритеты.
- Планировщик управляет вычислительными ресурсами и разделяет их между всеми заданиями.
- Существует много разных алгоритмов планирования, которые может использовать планировщик, но все они должны соответствовать определенным требованиям.
- В зависимости от выбранной стратегии планировщик должен либо выделить определенный временной интервал, на протяжении которого задание может использовать вычислительный блок, либо подождать, пока блок не будет освобожден.
- Если стратегия планирования вытесняющая, то у планировщика должна быть возможность принудительно освобождать ядро процессора, чтобы отдать его следующему заданию. Это называется вытесняющим планированием. Есть еще одна стратегия, в которой задание добровольно освобождает процессор, — так называемое кооперативное планирование.
- Алгоритмы вытесняющего планирования пытаются разделять временные интервалы равномерно и беспристрастно между разными заданиями.



Процессы и потоки

Задание — абстрактная концепция общего характера, которая может означать любой набор инструкций, предназначенный для выполнения в конкурентной системе, не обязательно компьютерной.

В ОС роль заданий играют либо процессы, либо потоки. Большинство операционных систем обращаются с ними практически одинаковым образом: как с некими заданиями, которые необходимо выполнять конкурентно.

Операционная система должна использовать планировщик в целях разделения ядер процессора между множеством заданий, будь то процессы или потоки, которым для выполнения нужно процессорное время. В момент создания новый процесс или поток поступает в очередь планировщика в качестве нового задания и, прежде чем начинать работу, ждет, когда ему выделят процессорное ядро.

С этого момента мы будем подразумевать, что все упоминаемые планировщики являются вытесняющими. Иные ситуации будут оговорены отдельно.

В ходе выполнения задание может испытывать сотни и даже тысячи переключений контекста (планировщики вытесняющие — переключают принудительно процессы). Однако эти переключения имеют одно причудливое и уникальное свойство — они непредсказуемы. Мы не знаем, когда или даже на какой инструкции контекст будет переключен! Даже в двух максимально приближенных прогонах программы на одной и той же платформе переключения контекста будут отличаться. Переключения контекста крайне непредсказуемы, поэтому лучше всего предполагать, что они с одинаковой вероятностью могут произойти на любой инструкции.



Порядок выполнения инструкций

```
Task P {
  1. num = 5
  2. num++
  3. num = num - 2
  4. x = 10
  5. num = num + x
}
```

Run 1:

```
1. num = 5
2. num++
>>>> Context Switch <<<<
3. num = num - 2
4. x = 10
>>>> Context Switch <<<<
5. num = num + x
```

Простое задание с пятью инструкциями

Инструкции пронумерованы; это значит, каждая следующая инструкция может начинать работу только после завершения предыдущей. Инструкция `num++` должна быть выполнена перед `num = num - 2`, и это ограничение должно соблюдаться независимо от того, как переключается контекст.

Потенциальные прогоны задания с переключениями контекста

Run 2:

```
num = 5
>> Context Switch <<
num++
num = num - 2
>> Context Switch <<
x = 10
>> Context Switch <<
num = num + x
```

Общее поведение отдельно взятого задания может быть детерминированным. Какими бы ни были переключения контекста в разных прогонах, общее состояние задания остается неизменным.

Благодаря необходимости соблюдать порядок выполнения инструкций конкурентность не может повлиять на общее состояние задания.

В конкурентной системе с несколькими заданиями, каждое из которых может читать и записывать разделяемый ресурс, разные прогоны будут давать разные результаты.

Когда следует использовать конкурентность

Любая программа, независимо от языка программирования, представляет собой набор инструкций, которые должны выполняться последовательно. Обычно говорят, что текущая инструкция блокирует следующую; в таком случае текущую инструкцию иногда называют блокирующей.

В любой программе **все инструкции — блокирующие**, а каждый **поток выполнения — последователен**.

Как определить, требуется ли программе конкурентность?

Если говорить более подробно, то каждой блокирующей инструкции требуется какое-то время на завершение работы. С нашей точки зрения, лучше всего, когда заданная инструкция завершается относительно быстро, не задерживая поток выполнения. Но нам не всегда так везет. В некоторых ситуациях невозможно определить, сколько времени понадобится блокирующей инструкции. Обычно это происходит, когда она ожидает какого-то события или доступа к неким данным.

Примеры: клиент-сервер; ввод пользователя

Разделив наш изначальный поток выполнения на две задачи, мы можем сделать так, чтобы одна задача ждала завершения блокирующей инструкции, а другая продолжала выполнять следующие инструкции, которые блокировались в нашей предыдущей неконкурентной конфигурации.



Когда следует использовать конкурентность

Пример, на котором мы сконцентрируемся, посвящен серверной программе, которая имеет три обязанности:

- получает от клиента два числа и возвращает ему их сумму;
- регулярно записывает в файл количество обслуженных клиентов, независимо от того, занимается она обслуживанием клиента в этот момент или нет;
- вдобавок должна уметь обслуживать сразу несколько клиентов.

```
Calculator Server {  
  Task T1 {  
    1. N = 0  
    2. Prepare Server  
    3. Do Forever {  
      4.   Wait for a client C  
      5.   N = N + 1  
      6.   Read the first number from C and store it in X  
      7.   Read the second number from C and store it in Y  
      8.   Z = X + Y  
      9.   Write Z to C  
      10.  Close the connection to C  
      11.  Write N to file  
    }  
  }  
}
```



Когда следует использовать конкурентность

```
Calculator Server {  
  Shared Variable: N  
  Task T1 {  
    1. N = 0  
    2. Prepare Server  
    3. Spawn task T2  
    4. Do Forever {  
    5.   Write N to file  
    6.   Wait for 30 seconds  
    }  
  }  
  Task T2 {  
    1. Do Forever {  
    2.   Wait for a client C  
    3.   N = N + 1  
    4.   Spawn task T3 for C  
    }  
  }  
  Task T3 {  
    1. Read first number from C and store it in X  
    2. Read first number from C and store it in Y  
    3. Z = X + Y  
    4. Write Z to C  
    5. Close the connection to C  
  }  
}
```

Серверная программа с тремя конкурентными заданиями

если у вас есть блокирующая операция, время выполнения которой либо слишком длинное, либо непредсказуемое, то вы должны разбить задание на два конкурентных потока.

Когда следует использовать конкурентность

Дальше может возникнуть вопрос: когда завершаются задания? Мы знаем, что задание заканчивает работу, выполнив все свои инструкции. Тем не менее если у нас есть бесконечный цикл, включающий в себя все инструкции, то задание не останавливается и время его жизни зависит от родительского задания, которое его породило.

Редко, но иногда все же возникает ситуация, когда на выполнение всех операций чтения уходит слишком много времени (намеренно или случайно), а количество новых клиентов быстро растет, и в какой-то момент получается множество экземпляров задания ТЗ, которые ждут от своих клиентов входные числа.

Описанная выше ситуация, когда серверная программа перестает принимать соединения от клиентов, называется отказом в обслуживании (denial of service, DoS). В процессе DoS-атаки оказывается чрезмерная нагрузка на серверный компьютер, чтобы он вышел из строя и прекратил отвечать на запросы. DoS-атаки принадлежат к группе сетевых атак, которые пытаются блокировать определенные сервисы с целью сделать их недоступными для клиентов. В их число входит широкий спектр вредоносных методов, нацеленных на остановку обслуживания, включая эксплойты. Это даже может выражаться в флудинге сети в попытке вывести из строя ее инфраструктуру.



Разделяемые состояния

Термин **«состояние»** означает набор переменных и их соответствующих значений в определенный момент времени. Следовательно, когда речь идет об итоговом состоянии, имеется в виду набор всех имеющихся неразделяемых переменных и их значений непосредственно после выполнения последней инструкции задания.

Промежуточное состояние задания — набор всех существующих неразделяемых переменных и их значений в момент выполнения определенной инструкции. Следовательно, на каждой инструкции промежуточное состояние задания меняется и количество таких состояний равно количеству инструкций. Согласно нашим определениям, последнее промежуточное состояние задания совпадает с итоговым.

Разделяемое состояние — тоже набор переменных вместе с соответствующими значениями в определенный момент времени, но эти значения могут быть прочитаны и изменены системой конкурентных заданий. Разделяемое состояние не принадлежит конкретному заданию (не является его локальным состоянием); любое задание, запущенное в системе, может прочитать и перезаписать его в любой момент.

В целом нас не интересуют разделяемые состояния, доступные только для чтения. Обращение к ним со стороны множества конкурентных заданий обычно является безопасным и не создает никаких проблем. Но если должным образом не защитить разделяемое состояние, которое можно изменять, то это может иметь серьезные отрицательные последствия. Поэтому все разделяемые состояния, рассматриваемые в этом разделе, доступны для записи как минимум одному заданию.

Разделяемые состояния

Система с двумя конкурентными заданиями и изменяемым разделяемым состоянием

```
Concurrent System {
  Shared State {
    X : Integer = 0
  }
  Task P {
    A : Integer
    1. A = X
    2. A = A + 1
    3. X = A
    4. print X
  }
  Task Q {
    B : Integer
    1. B = X
    2. B = B + 2
    3. X = B
    4. print X
  }
}
```

Возможное чередование заданий P и Q в ходе конкурентного выполнения

Задание P	Планировщик заданий	Задание Q

	Переключение контекста	
		B = X
		B = B + 2
	Переключение контекста	
A = X	Переключение контекста	
		X = B
	Переключение контекста	
A = A + 1		
X = A	Переключение контекста	
		print X
	Переключение контекста	
print X		
	Переключение контекста	

Разделяемые состояния

Система с двумя конкурентными заданиями и изменяемым разделяемым состоянием

```
Concurrent System {
  Shared State {
    X : Integer = 0
  }
  Task P {
    A : Integer
    1. A = X
    2. A = A + 1
    3. X = A
    4. print X
  }
  Task Q {
    B : Integer
    1. B = X
    2. B = B + 2
    3. X = B
    4. print X
  }
}
```

Возможное чередование заданий P и Q в ходе конкурентного выполнения

Задание P	Планировщик заданий	Задание Q
	-----	-----
	Переключение контекста	B = X
		B = B + 2
		X = B
	Переключение контекста	
A = X		
A = A + 1		
	Переключение контекста	
	Переключение контекста	print X
X = A		
print X		
	Переключение контекста	

Разделяемые состояния

Планировщик и принципе его работы.

Если у нас есть всего одно процессорное ядро, то в любой отдельно взятый момент оно может выполнять только одно задание.

Мы также знаем, что планировщик заданий сам по себе является компонентом программы, которому для выполнения тоже нужно занимать ядро процессора. Каким же образом он управляет разными заданиями, если ядро уже занято? Представьте, что планировщику выделено процессорное время. Вначале он выбирает задание из очереди, затем устанавливает таймер, по которому происходит прерывание, и освобождает ядро процессора, передавая ресурсы выбранному заданию.

Мы исходим из того, что планировщик выделяет каждому заданию определенный промежуток времени, по истечении которого произойдет прерывание. В результате ядро процессора перестанет выполнять текущее задание и немедленно загрузит планировщик, который сохранит последнее состояние предыдущего задания и загрузит следующее из очереди. Все это происходит, пока не загрузится операционная система. На компьютере с многоядерным процессором ситуация может быть другой: операционная система может планировать задания не на тех ядрах, на которых выполняется сама.

Адженда

Конкурентность

30 минут

**Синхронизация
поток**

30 минут

Синхронизация

30 минут

Процессы

30 минут

**Многопоточное
выполнение**

30 минут

**Синхронизация
процессов**

30 минут

Проблемы с конкурентностью

Виды проблем, которые могут возникнуть применительно к конкурентности:

- **Естественные проблемы.** Проблемы, существующими в конкурентной системе, в которой нет механизма управления (синхронизации): разные варианты чередования инструкций приводят к разным итоговым состояниям.
- **Постсинхронизационные проблемы.** Проблемы, возникшими в результате попытки исправления проблем из первой группы.



Естественные проблемы

Любая конкурентная система с несколькими заданиями имеет ряд возможных вариантов чередования.

Чередования сами по себе нельзя назвать проблемными. Но в некоторых случаях оно нарушает определенные ограничения. Именно в таких ситуациях чередования вызывают проблемы.

При конкурентном выполнении ряда заданий количество чередований может быть большим. Однако проблемы возникают, только когда ограничения системы, которые должны оставаться неизменными, варьируются между разными прогонами. Поэтому наша цель — внедрить такие управляющие механизмы (их иногда называют механизмами синхронизации), которые бы сохраняли ограничения в их исходном виде.

У любой конкурентной системы есть по крайней мере некоторые четко определенные инвариантные ограничения.

Чередования, происходящие в конкурентной системе, должны соответствовать заранее определенным инвариантным ограничениям. В противном случае с системой что-то не так. Если чередование нарушает инвариантное ограничение системы, то это значит, что мы имеем дело с состоянием гонки.

Состояние гонки — следствие неотъемлемого свойства конкурентной системы: чередований. При его возникновении всегда возникает опасность нарушения инвариантного ограничения. Последствия подобных нарушений могут выражаться как в логических ошибках, так и в сбоях.



Естественные проблемы

Система с тремя конкурентными заданиями, которые работают с одной разделяемой переменной

```
Concurrent System {
  Shared State {
    Counter : Integer = 0
  }
  Task T1 {
    A : Integer
    1.1. A = Counter
    1.2. A = A + 1
    1.3. Counter = A
  }
  Task T2 {
    B : Integer
    2.1. B = Counter
    2.2. B = B + 1
    2.3. Counter = B
  }
  Task T3 {
    A : Integer
    3.1. A = Counter
    3.2. A = A + 1
    3.3. Counter = A
  }
}
```

Чередования, которые нарушают инвариантное ограничение

Планировщик заданий	Задание T1	Задание T2	Задание T3

Переключение контекста	A = Counter A = A + 1 Counter = A		
Переключение контекста		B = Counter B = B + 1	
Переключение контекста			A = Counter
Переключение контекста		Counter = B	
Переключение контекста			A = A + 1 Counter = A

возникновение логической ошибки в результате состояния гонки
Инструкции 2.3 и 3.3 сохраняют в переменную Counter значение 2. Эта ситуация называется гонкой данных

Естественные проблемы

Чередование, нарушающее инвариантное ограничение

```

Concurrent System {
    Shared State {
        char *ptr = NULL;
// Разделяемый указатель типа char
// должен указывать на адрес
// в пространстве кучи. Он становится
// равным null по умолчанию.
    }
    Task P {
        1.1. ptr = (char*)malloc(10 * sizeof(char));
        1.2. strcpy(ptr, "Hello!");
        1.3. printf("%s\n", ptr);
    }
    Task Q {
        2.1. free(ptr);
        2.2. ptr = NULL;
    }
}

```

Одно из очевидных инвариантных ограничений, которое интересует нас в этом примере и не было определено явно, состоит в том, чтобы не допустить сбой задания. Если задание не может доработать до конца, то наличие инвариантного ограничения само по себе является противоречивым.

Существует несколько чередований, которые могут привести к сбою задания. Рассмотрим два из них.

Представьте, что первой выполняется инструкция 2.1. Поскольку указатель ptr равен null, задание Q аварийно завершится, а задание P продолжит работу

Еще один вариант чередования происходит, когда инструкция 2.2 выполняется до 1.2, но после 1.1. В этом случае сбой произойдет в задании P, а задание Q продолжит работу без каких-либо проблем.

Естественные проблемы

Наличие состояния гонки в конкурентной системе может иметь разные последствия, такие как логические ошибки и неожиданные сбои. Очевидно, что обе проблемы нуждаются в адекватном решении.

Следует отдельно отметить: не все состояния гонки в конкурентной системе можно обнаружить легко. Некоторые из них проявляются только со временем.

Тем не менее обнаружить состояния гонки на участках кода, которые выполняются не так часто, иногда можно с помощью специальных инструментов. На самом деле они позволяют обнаружить чередования, приводящие к состоянию гонки.

Состояние гонки можно обнаружить с помощью отдельной категории программ, которые делятся на **статические** и **динамические**. Статические инструменты анализируют исходный код, пытаясь сгенерировать из имеющихся инструкций все возможные чередования. Динамические же сначала запускают программу и ждут, когда она выполнит код, который может привести к состоянию гонки. Оба подхода используются совместно, что позволяет минимизировать риск возникновения подобных проблем.

Можно сделать следующий вывод: чтобы удовлетворить инвариантные ограничения, некоторые инструкции, принадлежащие разным заданиям, должны быть выполнены в строго определенном порядке, независимо от варианта чередований.

Следовательно, чередования, соответствующие этому порядку, не нарушают инвариантное ограничение. Нас они устраивают, и мы можем наблюдать нужный вывод. Чередования, которые не обеспечивают строгий порядок выполнения, нарушают инвариантное ограничение и могут считаться проблемными.

Естественные проблемы

Очень простая конкурентная система, страдающая от состояний гонки

```
Concurrent System {  
  Shared State {  
    X : Integer = 0  
  }  
  Task P {  
    1.1. X = 1  
  }  
  Task Q {  
    2.1. print X  
  }  
}
```

На случай чередований нам нужны механизмы, которые позволят восстановить порядок выполнения и убедиться в том, что инвариантные ограничения всегда соблюдаются.

Согласно ограничению, которое должно оставаться неизменным, программа должна выводить 1.

Этот пример может иметь два разных вывода в зависимости от типа чередований. Если нам нужно вывести 1, что соответствует инвариантному ограничению, то мы должны определить строгий порядок выполнения инструкций в двух разных заданиях.

В связи с этим инструкция print под номером 2.1 должна выполняться только после инструкции 1.1. Существует еще одно чередование, которое легко нарушает данный порядок (а значит, и инвариантное ограничение), и потому мы имеем состояние гонки. Нам нужно определить строгий порядок выполнения этих инструкций. Но сделать это не так-то просто.

Естественные проблемы

простая система, которая страдает от состояния гонки, однако не имеет разделяемого состояния

```
Concurrent System {  
  Shared State {  
  }  
  Task P {  
    1.1. print 3  
  }  
  Task Q {  
    2.1. print 1  
  }  
  Task R {  
    3.1. print 2  
  }  
}
```

В коде описана система с тремя заданиями. Следует отметить, что у нее нет разделяемого состояния. Но, несмотря на это, мы получаем состояние гонки. Определим для данной системы инвариантное ограничение: сначала должно выводиться число 1, затем 2 и в конце 3.

Даже в этой крайне простой системе нельзя предсказать, какое задание будет первым, вследствие чего мы имеем состояние гонки. Следовательно, чтобы не нарушить инвариантное ограничение, инструкции должны выполняться в таком порядке: 2.1, 3.1, 1.1.

Важное свойство состояний гонки: их возникновение в конкурентной системе не требует разделяемого состояния. Чтобы их избежать, некоторые инструкции должны всегда выполняться в строго определенном порядке. Следует отметить: состояние гонки встречается только когда небольшая группа инструкций, известная под названием «критический участок», выполняется не по порядку, в то время как другие инструкции могут выполняться в любом порядке. Если у вас есть изменяемое разделяемое состояние с инвариантным ограничением, то операции чтения и записи в отношении данного состояния, возможно, придется выполнять в строгом порядке. Это просто означает, что у всех заданий всегда должна быть возможность прочесть последнее, самое свежее значение разделяемого состояния. Кроме того, прежде чем приступить к его изменению, они должны знать о любых обновлениях, произошедших с момента чтения.

Естественные проблемы

Конкурентная система,
страдающая от состояния
гонки за разделяемой
переменной X

```
Concurrent System {
  Shared State {
    X : Integer = 2
  }
  Task P {
    A : Integer
    1.1. A = X
    1.2. A = A + 1
    1.3. X = A
  }
  Task Q {
    B : Integer
    2.1. B = X
    2.2. B = B + 3
    2.3. X = B
  }
}
```

Выражение $X = X + 1$ (в C записывается как $X++$) не атомарно. То есть его нельзя уместить в одну инструкцию. Это связано с тем, что у нас не может быть прямого доступа к памяти, когда мы выполняем с ней некую операцию. Мы всегда используем временную переменную (или регистр процессора) для хранения последнего значения. Поэтому операция выполняется с данной переменной или регистром, и только потом результат передается обратно в память. Следовательно, как бы вы это ни записывали, у вашего задания всегда будет временная локальная переменная.

В системах с несколькими ядрами процессора ситуация только усугубляется. Процессор также имеет кэш, в котором хранятся переменные, и значения из него попадают в основную память не мгновенно.

Каждый раз, когда чередование нарушает ограничение целостности данных, относящееся к разделяемой переменной, эта переменная начинает участвовать в гонке данных.

Гонки данных очень похожи на состояния гонки, но их возникновение требует наличия переменной, разделяемой между несколькими заданиями, и как минимум одно из этих заданий должно иметь возможность ее изменять. Иными словами, разделяемая переменная должна быть доступна не только для чтения, и по меньшей мере одному заданию следует иметь логику, которая может что-то записать в эту переменную.

Естественные проблемы

Конкурентная система с разделяемой переменной, доступной только для чтения

```

Concurrent System {
  Shared State {
    X : Integer (read-only) = 5
  }
  Task P {
    A : Integer
    1.1. A = X
    1.2. A = A + 1
    1.3. print A
  }
  Task Q {
    2.1. print X
  }
  Task R {
    B : Integer
    3.1. B = X + 1
    3.2. B = B + 1
    3.3. print B
  }
}

```

У переменной, доступной только для чтения, не может быть гонки данных. Целостность такой переменной невозможно нарушить, поскольку ее нельзя изменить.

В примере продемонстрировано, как состояние гонки может возникнуть в ситуации, в которой невозможна гонка данных, относящаяся к переменной, доступной только для чтения.

Представим, что в этом примере действует следующее инвариантное ограничение: поддерживать целостность данных и выводить сначала 5, затем 6 и в конце 7. Очевидно, мы имеем состояние гонки, так как должны обеспечить строгий порядок выполнения разных инструкций print.

Но, поскольку разделяемую переменную можно только читать, гонки данных не возникает. **Обратите внимание:** инструкции 1.2 и 3.2 изменяют лишь свое локальное состояние, что нельзя считать обновлением разделяемой переменной.

Не надейтесь легко устранить состояния гонки! Чтобы обеспечить определенный порядок выполнения инструкций из разных заданий, вам, несомненно, понадобятся некие механизмы синхронизации.

Постсинхронизационные проблемы

Ключевые проблемы, которые можно ожидать в результате неправильного использования управляющих механизмов:

Новые естественные проблемы. Применение управляющих механизмов может привести к разным состояниям гонки или гонкам данных. Эти механизмы пытаются обеспечить определенный порядок следования инструкций, из-за чего могут возникнуть новые естественные проблемы.

Недоступность ресурсов. Эта ситуация возникает, когда задание в конкурентной системе не может обратиться к разделяемому ресурсу на протяжении длительного времени (что обычно случается из-за применения определенного управляющего механизма).

Взаимные блокировки. Когда все задания в конкурентной системе ждут друг друга и ни одно из них не продвигается вперед, мы называем подобную ситуацию взаимной блокировкой. Так происходит в основном в результате неправильного применения управляющего механизма, что заставляет задания войти в бесконечный цикл, в котором они ждут, когда одно из них освободит разделяемый ресурс или снимет блокировку.

Перестановка приоритетов. Иногда после применения метода синхронизации задание с приоритетным доступом к разделяемому ресурсу блокируется заданием с низким приоритетом. То есть их приоритеты меняются местами.

Изначально конкурентная система не страдает от недоступности ресурсов. Если к планировщику операционной системы не применять никаких методов синхронизации, то он будет вести себя беспристрастно и не оставит ни одно задание без доступа к разделяемому ресурсу. Эта проблема может возникнуть, только если программист внедрил некие управляющие механизмы.

Методы синхронизации

Методы синхронизации, известны также как методы управления конкурентностью или конкурентные управляющие механизмы. Они предназначены для решения естественных проблем, присущих конкурентным системам. Если говорить в контексте всего того, что мы обсудили ранее, то управляющие механизмы пытаются нивелировать проблемы, которые могут возникнуть в системе из-за определенных чередований.

У любой конкурентной системы есть инвариантные ограничения, и не все чередования будут соблюдать каждое из них. Для таких чередований необходимо придумать механизм, который будет обеспечивать определенный порядок следования инструкций. Иными словами, нам нужно создать новые чередования, не нарушающие инвариантные ограничения, и подставить их вместо «плохих» чередований. После применения определенного метода синхронизации мы получим совершенно новую конкурентную систему с некоторыми новыми чередованиями. Все это делается в надежде на то, что новая система будет соблюдать инвариантные ограничения и не вызовет никаких постсинхронизационных проблем.

Обратите внимание: чтобы использовать метод синхронизации, нам придется писать новый код и изменять имеющийся. В последнем случае вы фактически меняете порядок выполнения инструкций и, следовательно, чередования. Этим вы, в сущности, создаете новую конкурентную систему с новыми чередованиями.



Методы синхронизации

Каким образом наличие новых чередований решает наши проблемы с конкурентностью? Добавляя новые чередования, мы ужесточаем порядок следования инструкций из разных заданий, что обеспечивает соответствие инвариантным ограничениям.

Стоит отметить, что две инструкции в одном задании всегда размещены в определенном порядке, чего нельзя сказать об инструкциях из разных заданий в конкурентной системе. Этот порядок мы устанавливаем за счет использования методов синхронизации.

Совершенно новая конкурентная система несет в себе новые проблемы, отличные от старых. Исходная конкурентная система носила естественный характер, и планировщик заданий был единственным ее компонентом, который инициировал переключения контекста. Но после внесения изменений мы получили искусственную конкурентную систему, помимо планировщика заданий имеющую и другие действующие элементы. Важную роль в ней играют механизмы управления конкурентностью, которые используются для сохранения ее инвариантных ограничений. В результате проявляются новые, постсинхронизационные проблемы, которые мы обсуждали в предыдущем разделе.

Выбор подходящих управляющих методик в целях такой синхронизации разных заданий, чтобы они соблюдали определенный порядок выполнения, зависит от исходного конкурентной среды. Например, управляющие механизмы, которые используются в многопроцессной программе, могут отличаться от методов, применяемых в многопоточном приложении.

Холостые циклы и циклические блокировки

Чтобы заставить инструкцию из одного задания выполняться после инструкции из другого, первое задание должно подождать, пока второе не выполнит свою инструкцию. Тем временем если первое получит доступ к процессору в результате переключения контекста, то не должно продолжать работу. Иными словами, оно должно остановиться и подождать, пока не будет выполнена инструкция из второго задания. Это обобщенное решение.

После того как второе задание выполняет свою инструкцию, перед нами открываются две возможности. Первое задание может само проверить, завершилась ли эта инструкция, или же второе задание может уведомить первое о том, что оно может продолжать работу.

Этот сценарий похож на ситуацию, когда два человека пытаются что-то сделать в определенном порядке. Один из них должен ждать, пока второй не совершит свои действия, и только потом берется за работу. Можно сказать, что почти все управляющие механизмы используют аналогичный подход, но их реализации могут отличаться и зависеть от механизмов, доступных в определенной среде.



Холостые циклы и циклические блокировки

```
Concurrent System {  
  Task P {  
    1.1. print 'A'  
  }  
  Task Q {  
    2.1. print 'B'  
  }  
}
```

Очевидно, мы имеем состояние гонки, основанное на инвариантном ограничении. Чередование {2.1, 1.1} выводит сначала A, а затем B, а это противоречит данному ограничению. Следовательно, чтобы сохранить определенный порядок выполнения этих инструкций, нам нужен управляющий механизм.

```
Concurrent System {  
  Shared State {  
    Done : Boolean = False  
  }  
  Task P {  
    1.1. print 'A'  
    1.2. Done = True  
  }  
  Task Q {  
    2.1. While Not Done Do Nothing  
    2.2. print 'B'  
  }  
}
```

Холостой цикл — хоть и неэффективный, но зато простой способ ожидания событий. Поскольку внутри него задание не может делать ничего особенного, он расходует выделенный ему интервал без какой-либо пользы. Холостых циклов избегают при продолжительном ожидании.



Механизм ожидания/уведомления

Задание Q может не ждать флаг Done, а просто «уснуть», а задание P может его уведомить, когда этот флаг станет равен True.

```
Concurrent System {  
    Task P {  
        1.1. print 'A'  
        1.2. Notify Task Q  
    }  
    Task Q {  
        2.1. Go To Sleep Mode  
        2.2. print 'B'  
    }  
}
```

Каково преимущество этого подхода?

«Спящее» задание не входит в холостой цикл и не тратит процессорное время. Вместо того чтобы запускать новый холостой цикл для проверки условия, оно «засыпает» и по выполнении условия получает уведомление. Это существенно повышает эффективность использования процессора, и задания, которым действительно нужны ресурсы, смогут их получить.

Когда задание переходит в режим сна, у вас должна быть возможность как-то его «пробудить». Для этого заданию обычно отправляются уведомления или сигналы. Когда уведомленное задание «просыпается», планировщик снова помещает его в очередь и выделяет ему ресурсы процессора. Затем задание продолжит работу непосредственно с того места, на котором «уснуло».

Благодаря этому методу мы не используем холостой цикл и не тратим время процессора впустую. **Стоит отметить:** для «засыпания» и уведомления «спящего» задания в большинстве операционных систем (особенно в POSIX-совместимых) предусмотрены специальные системные вызовы.

Механизм ожидания/уведомления

Вариант чередования, вызывающий в конкурентной системе частичную взаимную блокировку

```
1.1. print 'A'
1.2. Notify Task Q
2.1. Go To Sleep Mode
2.2. print 'B'
```

Решения, основанные на механизме ожидания/уведомления, обычно реализуются с помощью условных переменных. Ни один механизм синхронизации не обходится без ожидания. Это единственный способ, с помощью которого задания могут оставаться синхронизированными.

В этом сценарии задание Р вывело А и затем уведомило задание Q, которое еще не успело «заснуть», поскольку ему не было выделено процессорное время. Получив ресурсы процессора, Q сразу же «засыпает». Однако у нас нет никаких других заданий, которые могли бы его уведомить. Следовательно, задание Q больше не будет выполняться ввиду того, что планировщик не выделяет ядра процессора «спящим» заданиям.

```
Concurrent System {
  Shared State {
    Done : Boolean = False
  }
  Task P {
    1.1. print 'A'
    1.2. Done = True
    1.3. Notify Task Q
  }
  Task Q {
    2.1. While Not Done {
    2.2. Go To Sleep Mode If Done is False (Atomic)
    2.3. }
    2.4. print 'B'
  }
}
```

Семафоры и мьютексы

В 1960-е годы Эдсгер Дейкстра, известнейший датский математик и ученый в области информационных технологий, вместе со своей командой разработал новую операционную систему THE Multiprogramming System или THE OS для компьютера Electrologica X8. Система имела уникальную по тем временам архитектуру. Это произошло менее чем за десять лет до того, как компания Bell Labs создала Unix и позже язык C. Для написания THE OS использовался ассемблер. Это была многозадачная операционная система и имела многоуровневую архитектуру. На самом верхнем уровне находился пользователь, а на самом нижнем — планировщик заданий. В терминах Unix это было эквивалентно совместному размещению планировщика заданий и диспетчера управления процессами в кольце ядра. Одной из идей, которую Дейкстра придумал вместе со своей командой, чтобы преодолеть некоторые сложности, связанные с конкурентностью, а также разделить ресурсы между разными заданиями, была концепция семафоров.

Семафоры — обычные переменные или объекты, которые используются для синхронизации доступа к разделяемому ресурсу.

Когда задание хочет обратиться к разделяемому ресурсу (это может быть обычная переменная или файл), оно должно сначала проверить семафор и попросить разрешения продолжить работу.

Критический участок — всего лишь набор инструкций, защищенных семафором. Задание не может в него войти без разрешения от семафора. С другой стороны, задача семафора — защищать критический участок. Каждый раз, пытаясь войти в него, задание должно уведомить об этом соответствующий семафор.

Точно так же этот семафор должен быть уведомлен о том, что задание завершило работу и хочет покинуть критический участок.

Семафоры и мьютексы

```

Concurrent System {
  Shared State {
    S : Semaphore which allows only 1 task at a time
    Counter: Integer = 0
  }
  Task P {
    A : Local Integer
    1.1. EnterCriticalSection(S)
    1.2. A = Counter
    1.3. A = A + 1
    1.4. Counter = A
    1.5. LeaveCriticalSection(S)
  }
  Task Q {
    B : Local Integer
    2.1. EnterCriticalSection(S)
    2.2. B = Counter
    2.3. B = B + 2
    2.4. Counter = B
    2.5. LeaveCriticalSection(S)
  }
}

```

Использование семафоров для синхронизации заданий

В нашем примере семафор S используется таким образом, что на его критический участок может зайти только одно задание. Но семафоры куда более универсальны; они могут позволить находиться на критических участках сразу нескольким заданиям (количество определяется в момент создания семафора). Семафор, который разрешает входить на критический участок по одному, обычно называют двоичным семафором или мьютексом. Мьютексы куда более распространены, чем семафоры общего вида, и их всегда можно встретить в конкурентном коде. API POSIX предоставляет как семафоры, так и мьютексы, и вы можете использовать их в зависимости от ситуации.

Семафоры и мьютексы

Термин **«мьютекс»** означает *mutual exclusion* (взаимное исключение). Представьте, что у нас есть два задания, каждое из которых имеет критический участок с доступом к одному и тому же разделяемому ресурсу. Чтобы получить решение на основе взаимного исключения, которое делает невозможным состояние гонки, наши задания должны отвечать следующим требованиям:

- в любой момент времени на критическом участке может находиться только одно из них, а другое должно ждать, пока критический участок не освободится;
- решение не должно быть подвержено взаимной блокировке. Задание, которое останавливается перед критическим участком, должно в конечном счете в него войти. В ряде случаев ограничивается максимальное время ожидания (период состязания);
- задание, находящееся на критическом участке, не может быть убрано с него преждевременно, чтобы туда могло войти другое задание. Иными словами, решение должно быть совместным, а не вытесняющим.

Мьютексы позволяют разработать решение на основе взаимного исключения.

Обратите внимание: критические участки тоже должны удовлетворять похожим критериям. Они должны пропускать внутрь по одному заданию и не должны вызывать взаимную блокировку. Следует отметить, что семафоры тоже совместимы с последними двумя требованиями, но позволяют заходить на критический участок сразу несколькими заданиями.

Взаимное исключение — самый важный аспект конкурентности и доминирующий фактор в различных управляющих механизмах, существующих на сегодняшний день.

Семафоры и мьютексы называют блокируемыми объектами. Ожидание семафора и вход на критический участок — это эквивалент блокировки семафора. С другой стороны, выход с критического участка и обновления семафора — это эквивалент разблокировки семафора.

Семафоры и мьютексы

```

Concurrent System {
  Shared State {
    S : Semaphore which allows only 1 task at a time
    Counter: Integer = 0
  }
  Task P {
    A : Local Integer
    1.1. Lock(S)
    1.2. A = Counter
    1.3. A = A + 1
    1.4. Counter = A
    1.5. Unlock(S)
  }
  Task Q {
    B : Local Integer
    2.1. Lock(S)
    2.2. B = Counter
    2.3. B = B + 2
    2.4. Counter = B
    2.5. Unlock(S)
  }
}

```

Использование операций блокировки и разблокировки для работы с семафорами

Эта терминология используется в API POSIX.

Когда несколько заданий хотят зайти на критический участок, они пытаются заблокировать семафор, но только определенное их количество (зависящее от семафора) сможет получить блокировку и выполнить вход. Другие задания будут ждать своей очереди. Процесс ожидания блокировки семафора называется **состязанием**. Чем больше заданий, тем жестче состязание, а его продолжительность определяет, насколько задания замедляют свое выполнение.

Мьютексы — основной механизм для синхронизации некоторых конкурентных заданий.

Системы с несколькими вычислительными блоками

Если компьютерная система имеет всего один вычислительный блок (одноядерный процессор), то задания, пытающиеся обратиться к определенному адресу в основной памяти, всегда получают самое последнее и актуальное значение, даже будь этот адрес закэширован в ядре процессора. Сохранение определенных адресов памяти (даже вместе с изменениями, которые в них вносятся) внутри локального кэша ядра — распространенная практика. Она повышает производительность за счет уменьшения количества операций чтения и записи, направленных к основной памяти. При возникновении определенных событий ядро процессора синхронизирует внесенные изменения, сохраненные в кэше, с основной памятью. Эти локальные кэши существуют, даже если у вас несколько вычислительных блоков. Под таковыми мы понимаем либо многоядерный процессор, либо несколько процессоров с любым количеством ядер. Заметьте, что каждое ядро имеет собственный локальный кэш. Таким образом, когда два разных задания выполняются на двух разных ядрах процессора и обращаются к одному и тому же адресу в основной памяти, каждое ядро заносит значение данного адреса в свой локальный кэш. Если одно из заданий попытается выполнить запись в разделяемый адрес, то изменения проявятся только в его локальном кэше, а не в основной памяти или в кэше других ядер процессора.



Системы с несколькими вычислительными блоками

```
Concurrent System {
  Shared State {
    Done : Boolean = False
  }
  Task P {
    1.1. print 'A'
    1.2. Done = True
    1.3. Notify Task Q
  }
  Task Q {
    2.1. While Not Done {
      2.2. Go To Sleep Mode If Done is False (Atomic)
    2.3. }
    2.4. print 'B'
  }
}
```

Чтобы решить проблему, которая возникает из-за наличия разных локальных кэшей в каждом ядре процессора, используется **протокол когерентности памяти**. В соответствии с ним все задания, работающие на разных ядрах, видят в своих локальных кэшах одно и то же значение, даже если одно из ядер изменило его. То есть можно сказать, что адрес памяти виден всем вычислительным блокам. Соблюдение протокола когерентности памяти делает память видимой для всех заданий, выполняющихся в разных вычислительных блоках.

Существует такая вероятность: задание Q, получив доступ к процессору и прочитав свой локальный кэш, увидит, что переменная Done равна False, и перейдет в спящий режим, а задание P завершит работу и больше не будет слать уведомления (поскольку уже давно это сделало). В конечном счете Q «засыпает» навсегда. в результате чего происходит частичная взаимная блокировка.

Системы с несколькими вычислительными блоками

```
Concurrent System {  
  Shared State {  
    Done : Boolean = False  
  }  
  Task P {  
    1.1. print 'A'  
    1.2. Done = True  
    1.3. Memory Barrier  
    1.4. Notify Task Q  
  }  
  Task Q {  
    2.1. Do {  
      2.2. Memory Barrier  
      2.3. Go To Sleep Mode If Done is False (Atomic)  
    } While Not Done  
    2.5. print 'B'  
  }  
}
```

Улучшенное решение с использованием барьеров памяти

Чтобы решить эту проблему, необходимо использовать барьеры памяти. Это инструкции, которые ведут себя подобно барьерам; во время их выполнения (прохождения) все значения, измененные в одном локальном кэше, распространяются по основной памяти и других локальных кэшах. Они становятся видимыми для всех заданий, выполняющихся на других процессорных ядрах. Иными словами, барьеры памяти синхронизируют локальные кэши всех ядер процессора и основную память.

Системы с несколькими вычислительными блоками

```

Concurrent System {
  Shared State {
    Done : Boolean = False
    M : Mutex
  }
  Task P {
    1.1. print 'A'
    1.2. Lock(M)
    1.3. Done = True
    1.4. Unlock(M)
    1.5. Notify Task Q
  }
  Task Q {
    2.1. Lock(M)
    2.2. While Not Done {
      2.3. Go To Sleep Mode And Unlock(M) (Atomic)
      2.4. Lock(M)
    }
    2.5. }
    2.6. Unlock(M)
    2.7. print 'B'
  }
}

```

Улучшенное решение с

использованием мьютексов

Инструкции Lock(M) и Unlock(M) выступают барьерами памяти, гарантируя видимость переменных во всех заданиях.

Обратите внимание: когда задание блокирует мьютекс (или семафор), автоматическая разблокировка может произойти в трех случаях:

- задание разблокирует мьютекс с помощью команды Unlock;
- когда задание завершается, все заблокированные мьютексы разблокируются;
- когда задание входит в спящий режим, заблокированные мьютексы становятся разблокированными

Циклические блокировки

```
Concurrent System {  
    Shared State {  
        Done : Boolean = False  
        M : Mutex  
    }  
    Task P {  
        1.1. print 'A'  
        1.2. SpinLock(M)  
        1.3. Done = True  
        1.4. SpinUnlock(M)  
    }  
    Task Q {  
        2.1. SpinLock(M)  
        2.2. While Not Done {  
        2.3. SpinUnlock(M)  
        2.4. SpinLock(M)  
        2.5. }  
        2.6. SpinUnlock(M)  
        2.7. print 'B'  
    }  
}
```

Решение с использованием мьютекса и алгоритмов циклической блокировки

Пример должен подавать на стандартный вывод сначала A, а затем B.

Здесь используется мьютекс в сочетании с алгоритмом циклической блокировки. Мьютекс служит барьером памяти, поэтому у нас не будет никаких проблем с видимостью памяти; к тому же он фактически синхронизирует задания P и Q относительно разделяемого флага Done. Данный псевдокод можно перевести на любой язык программирования, поддерживающий конкурентность.

Мы используем циклические блокировки, которые представляют собой обычные алгоритмы холостого цикла. Каждый раз, когда вы блокируете такой мьютекс, задание входит в холостой цикл и ждет, пока тот не станет доступен, и только потом продолжает работу.

Циклические блокировки очень распространены в высокопроизводительных системах.



Условные переменные

```
Concurrent System {  
    Shared State {  
        Done : Boolean = False  
        CV : Condition Variable  
        M : Mutex  
    }  
    Task P {  
        1.1. print 'A'  
        1.2. Lock(M)  
        1.3. Done = True  
        1.4. Notify(CV)  
        1.5. Unlock(M)  
    }  
    Task Q {  
        2.1. Lock(M)  
        2.2. While Not Done {  
            2.3. Sleep(M, CV)  
        }  
        2.4. }  
        2.5. Unlock(M)  
        2.6. print 'B'  
    }  
}
```

Решение с использованием условной переменной

С помощью условных переменных мы сможем сделать так, чтобы задание подождало и получило уведомление.

Условными называются обычные переменные (или объекты), которые позволяют перевести задание в спящий режим или оповестить другие задания, чтобы те «проснулись».

Обратите внимание: под спящим режимом мы понимаем не задержку на какое-то определенное количество секунд или миллисекунд. Речь идет о том, что задание больше не хочет получать ресурсы процессора. По аналогии с мьютексами, которые защищают критический участок, условные переменные используются для организации обмена сигналами между разными заданиями. Условную переменную необходимо использовать в связке с мьютексом. Если этого не сделать, то вашему решению попросту будет не хватать взаимного исключения. Связку из мьютекса и условной переменной обычно называют **монитором**. Кроме того, существует одноименный шаблон проектирования, относящийся к конкурентности.

Конкурентность в POSIX

Поддержку конкурентности или многозадачности предоставляет ядро операционной системы. Не все ядра были конкурентными с момента своего появления, но большинство из них уже поддерживают конкурентность. Приятно отметить, что система Unix обзавелась этой возможностью вскоре после выхода первой версии. В POSIX и единой спецификации Unix пытались стандартизировать API, предоставляемый кольцом командной оболочки в Unix-подобных ОС. Конкурентность уже давно является частью этих стандартов и позволяет многим разработчикам писать конкурентные программы для POSIX-совместимых операционных систем. Поддержка конкурентности в POSIX реализована во множестве ОС, таких как Linux и macOS, и получила широкое применение. В POSIX-совместимых системах конкурентность обычно предоставляется двумя путями. Конкурентная программа может состоять либо из разных процессов (в этом случае ее называют многопроцессной), либо из разных потоков выполнения в рамках одного процесса (тогда ее именуют многопоточной).



Ядра с поддержкой конкурентности

Почти все ядра, разрабатываемые и сопровождаемые на сегодняшний день, — **многозадачные**. У любого ядра есть планировщик заданий, распределяющий вычислительные блоки процессора между множеством активных процессов и потоков.

При каждом запуске программы создается *новый процесс*, в котором выполняется ее логика. Процессы изолированы друг от друга; то есть один не имеет доступа к содержимому (например, к памяти) другого.

Похожим образом работают и потоки выполнения, но они находятся в рамках определенного процесса. С их помощью в процессе реализуется конкурентность; она достигается за счет того, что несколько потоков совместно выполняют разные наборы инструкций в конкурентной манере. Отдельно взятый поток нельзя разделить между двумя процессами; он всегда работает локально в том из них, которому принадлежит. Все потоки процесса-владельца имеют доступ к его памяти, работая с ней как с разделяемым ресурсом. В то же время каждый поток имеет собственный стек, доступный для других потоков того же процесса. Кроме того, и процессы, и потоки могут использовать разделяемые ресурсы центрального процессора, и планировщик заданий в большинстве операционных систем задействует один и тот же алгоритм для распределения между ними процессорных ядер.

В Unix-подобных ядрах термин «задание» обычно относится как к процессам, так и к потокам.

Ядра с поддержкой конкурентности

У Unix-подобных ядер есть планировщики, которые пытаются непредвзято управлять доступом к процессору со стороны различных заданий. В разных ядрах планировщики используют разные стратегии и алгоритмы для управления заданиями. Но большинство из них можно отнести к двум основным категориям:

- совместное планирование;
- вытесняющее планирование.

Совместное (или кооперативное) планирование состоит в том, что задание, которому выделили ядро процессора, должно добровольно его освободить.

Вытесняющее планирование — противоположность совместного. Оно позволяет заданию занимать ядро процессора до тех пор, пока его не освободит планировщик.

Существуют разные виды планирования с разделением времени, которые отличаются используемыми в них алгоритмами. Один из самых широко распространенных алгоритмов — **циклический перебор**. Он обеспечивает честный доступ к разделяемому ресурсу (в нашем случае это ядро процессора) для всех заданий без исключения.

Конкурентность можно воплотить в программном обеспечении двумя способами. Первый — **многопроцессность** — заключается в применении пользовательских процессов для выполнения параллельных заданий в многозадачной среде. Второй — **многопоточность** — подразумевает разбиение заданий на параллельные потоки выполнения в рамках одного процесса с помощью пользовательских потоков. В крупных программных проектах очень часто можно встретить сочетание этих подходов. Несмотря на то что оба способа делают ПО конкурентным, они отличаются по своей природе на фундаментальном уровне.

Многопроцессность

Многопроцессность — просто использование процессов для выполнения конкурентных заданий. Очень хороший пример — стандарт CGI (Common Gateway Interface — общий интерфейс шлюза) в веб-серверах. С его помощью веб-серверы запускают новый процесс интерпретатора для каждого HTTP-запроса. Таким образом, они могут обслуживать несколько запросов одновременно. Пример, Hadoop.

Данные процессы должны постоянно обмениваться фрагментами информации, чтобы кластер мог функционировать. Существует множество других распределенных многоузловых систем подобного рода, таких как Gluster, Kafka и криптовалютные сети. Все они опираются на внутреннее взаимодействие и обмен сообщениями между процессами, размещенными на разных узлах.

Когда процессы или потоки не имеют разделяемого состояния, между многопроцессностью и многопоточностью нет принципиальной разницы. Но при наличии разделяемого состояния разница между процессами, потоками и даже их сочетанием становится огромной. Одно из отличий состоит в том, какие методы синхронизации нам доступны. API для работы с этими механизмами более или менее одинаковые, но работать в многопроцессных средах куда сложнее, и их внутренняя реализация тоже различается, как и способы использования разделяемого состояния. В потоках можно задействовать все методики, которые доступны в процессах, но, помимо этого, им доступна такая роскошь, как возможность разделения состояния с помощью общей области памяти.

Если немного углубиться в подробности, то у процесса есть приватная память, которая доступна для чтения и записи только ему, поэтому ее не так-то просто использовать для разделения чего бы то ни было между разными процессами. А вот с потоками все намного проще. Все потоки внутри одного процесса имеют доступ к общей памяти, поэтому могут применять ее для хранения разделяемого состояния.

Многопроцессность

Механизмы, с помощью которых процессы могут обращаться к разделяемому состоянию. Больше информации о них будет дано в следующих главах.

- Файловая система.
- Отображение файла в память.
- Сеть.
- Сигналы.
- Разделяемая память.
- Каналы.
- Сокеты домена Unix.
- Очередь сообщений.
- Переменные среды.

В многопроцессных и многопоточных средах применяются очень похожие API, предоставляемые стандартом POSIX. Различия, скорее всего, заключаются во внутренней реализации мьютексов и условных переменных.



Многопоточность

Многопоточность — это когда для выполнения параллельных заданий в конкурентной среде применяются пользовательские потоки. Очень сложно найти реальную программу, состоящую из одного потока; почти все приложения, которые вам будут встречаться, являются многопоточными. Потоки могут существовать только внутри процессов; не бывает такого потока, у которого не было бы процесса-владельца. Каждый процесс содержит по меньшей мере один поток, обычно называемый главным или основным. Программа, которая выполняет все свои задания в одном потоке, называется однопоточной. Все потоки внутри одного процесса имеют доступ к одним и тем же областям памяти.

Поскольку у всех потоков есть свой стек, его можно задействовать для хранения разделяемых состояний. Один поток может передать другому указатель на какой-то участок своего стека, и последний сможет легко к нему обратиться ввиду того, что все эти адреса принадлежат стеку процесса. Поток также может легко работать с кучей, принадлежащей процессу, используя ее для хранения своих разделяемых состояний.

Механизмы синхронизации тоже очень напоминают те, которые применяются для процессов. Даже API POSIX ничем не отличается. Это, вероятно, объясняется тем фактом, что POSIX-совместимые операционные системы работают с процессами и потоками почти одним и тем же образом.

Microsoft Windows не поддерживает API POSIX для управления потоками (pthreads). Данная система имеет собственный API, который позволяет создавать потоки и управлять ими. Он входит в состав системной библиотеки Win32.

Адженда

Конкурентность

30 минут

**Синхронизация
потоков**

30 минут

Синхронизация

30 минут

Процессы

30 минут

**Многопоточное
выполнение**

30 минут

**Синхронизация
процессов**

30 минут

Потоки

Инициатор и постоянный владелец любого потока — **процесс**. Невозможно создать разделяемый поток или передать владением им другому процессу. У каждого процесса есть по меньшей мере один поток, который называют главным или основным. В программе на языке C в рамках главного потока выполняется функция `main`.

Все потоки имеют общий идентификатор процесса (`process ID`, `PID`) – можно проверить, если воспользоваться такой утилитой, как `top` или `htop`. Более того, каждый поток процесса наследует все его атрибуты, включая `ID` группы, пользователя, текущий рабочий каталог и обработчики сигналов. Например, поток имеет тот же текущий рабочий каталог, что и его процесс-владелец.

У каждого потока есть уникальный идентификатор (`thread ID`, `TID`). С его помощью можно отправлять сигналы конкретному потоку или отслеживать поток во время отладки. Как вы вскоре увидите, идентификатор POSIX-потока доступен в переменной `pthread_t`. Помимо этого, у каждого потока есть собственная выделенная ему маска сигналов, позволяющая фильтровать сигналы, которые он будет получать.

Все потоки внутри одного процесса имеют доступ ко всем файловым дескрипторам, открытым любым потоком в данном процессе. Следовательно, каждый поток может читать и изменять ресурсы, на которые ссылаются эти дескрипторы. То же самое касается дескрипторов открытых сокетов. Больше о дескрипторах сокетов и файлов вы узнаете далее в этой/следующей лекции.

К потокам можно применять все те методы разделения и передачи состояния.



Потоки

Методы, с помощью которых потоки могут разделять или передавать состояние в POSIX-совместимой системе:

- память процесса-владельца (сегменты данных, стека и кучи). Этот метод применим только к потокам, но не к процессам;
- файловая система;
- отображение файлов в память;
- сеть (с использованием сетевых сокетов);
- передача сигналов между потоками;
- разделяемая память;
- POSIX-каналы;
- сокеты домена Unix;
- очереди сообщений POSIX;
- переменные среды.

Продолжая тему свойств потоков, следует сказать, что все потоки внутри одного процесса могут использовать его память для хранения разделяемого состояния. Это самый распространенный способ разделения ресурсов между несколькими потоками. Для этого обычно применяется сегмент кучи процесса.

Время жизни потока зависит от времени жизни владеющего им процесса. Когда заканчивает работу главный поток, процесс немедленно завершается. Но если у него есть другие отсоединенные потоки, то он подождет, пока все они не доработают до конца.



Потоки

В качестве процесса, создающего поток, может выступать ядро. Но это может быть и пользовательский процесс, созданный в пользовательском пространстве. Если речь идет о ядре, то мы получаем так называемый поток уровня ядра (или просто поток ядра); в противном случае создается поток пользовательского уровня. Потоки ядра обычно выполняют важную логику, и в связи с этим имеют повышенный приоритет по сравнению с пользовательскими. Например, драйвер устройства может применять поток ядра в целях ожидания сигнала от оборудования.

По аналогии с тем, как пользовательские потоки имеют доступ к одной и той же области памяти, потоки ядра могут обращаться к пространству памяти этого ядра — то есть ко всем его процедурам и модулям.

Мы будем уделять внимание пользовательским потокам. Причина в том, что API для работы с ними предоставляется стандартом POSIX. А вот для создания потоков ядра и управления ими стандартного интерфейса нет; соответствующие механизмы реализуются конкретным ядром.

Создание и управление потоками ядра выходит за рамки лекции. Таким образом, начиная с этого момента, термин «поток» будет относиться только к пользовательским потокам.

Пользователь не может создать поток напрямую. Он должен сначала породить процесс, и только потом в главном потоке этого процесса можно будет инициировать еще один поток. Стоит отметить, что потоки могут создаваться лишь другими потоками.



Потоки

Что касается структуры памяти потоков, то у каждого из них есть стек, который можно считать приватной областью памяти, выделенной специально для него. Однако на практике к данной области могут обращаться и другие потоки внутри того же процесса; им для этого достаточно иметь указатель, ссылающийся на нее.

Следует помнить, что все эти сегменты стека являются частью общего пространства памяти процесса и доступны для всех его потоков.

Если говорить о методах синхронизации, то управляющие механизмы, с помощью которых синхронизируются процессы, можно использовать и для потоков. В число инструментов, позволяющих синхронизировать как потоки, так и процессы, входят семафоры, мьютексы и условные переменные.

Программу, в которой потоки синхронизированы и больше не наблюдается гонка данных или состояния гонки, называют потокобезопасной. Этот же термин применим к библиотеке или набору функций, которые можно легко использовать в многопоточной программе, не опасаясь появления новых проблем с конкурентностью. Мы как программисты должны стремиться к написанию потокобезопасного кода.

[По ссылке](#) вы можете найти информацию о POSIX-потоках и их общих характеристиках. Материал посвящен интерфейсу POSIX для работы с потоками в реализации NPTL. Обсуждение ведется в контексте среды Linux, но большая его часть применима и к другим Unix-подобным операционным системам.



POSIX-потоки

API POSIX для работы с потоками, более известный как библиотека pthread. Данный API очень важен, поскольку служит основным механизмом создания потоков и управления ими в POSIX-совместимых операционных системах.

В таких ОС, как Microsoft Windows, которые несовместимы с POSIX, для этого обычно предусмотрен другой API, и информацию о нем можно найти в документации соответствующей системы. Например, в случае с Microsoft Windows - [Win32 API](#).

Библиотека pthread — всего лишь набор заголовков и функций, пригодных для написания многопоточных программ в POSIX-совместимых операционных системах.

В качестве общеизвестного примера можно привести библиотеку потоков POSIX (Native POSIX Threading Library, NPTL) — главную реализацию pthread для операционной системы Linux.

Согласно API pthread, чтобы получить доступ ко всем функциям для работы с потоками, достаточно подключить заголовок pthread.h. У этой библиотеки также есть некоторые расширения, доступные только при подключении semaphore.h.

Библиотека потоков POSIX предоставляет следующие возможности:

- управление потоками, включая их создание, присоединение и отсоединение;
- мьютексы;
- семафоры;
- условные переменные;
- различные виды блокировок, включая циклически и рекурсивные.



Порождение POSIX-потоков

Порождение нового POSIX-потока *пример 30_1*

```
#include <pthread.h>
void *thread_body(void *arg) {
    printf("Hello from first thread!\n");
    return NULL;
}
int main(int argc, char **argv) {
    pthread_t thread; // Обработчик потоков
    // Создаем новый поток
    int result = pthread_create(&thread, NULL, thread_body, NULL);
    ...
    result = pthread_join(thread, NULL);
    ...
    return 0;
}
```

```
$ gcc main.c -o main.out -lpthread
$ ./main.out
Hello from first thread!
```

Непосредственно перед main мы объявили новую функцию: thread_body. У нее специфическая сигнатура. Она принимает один указатель void* и возвращает другой. void* — обобщенный тип указателя, который может представлять указатели любого другого типа, такого как int* или double*. Функция main — часть логики главного потока; она выполняется во время его создания.

Для компиляции необходимо добавить параметр -lpthread. Это делается для того, чтобы скомпоновать нашу программу с имеющейся реализацией библиотеки pthread. На некоторых платформах, таких как macOS, при компоновке программы можно обойтись и без параметра -lpthread.

Порождение POSIX-потоков

Пример с порождением отсоединенного потока, *пример 30_2*

```
#include <pthread.h>
void *thread_body(void *arg) {
    printf("Hello from first thread!\n");
    return NULL;
}
int main(int argc, char **argv) {
    pthread_t thread; // Обработчик потоков
    // Создаем новый поток
    int result = pthread_create(&thread, NULL, thread_body, NULL);

    result = pthread_detach(thread); // Отсоединяем поток

    pthread_exit(NULL); // Выходим из главного потока
    return 0;
}
```

```
$ gcc main.c -o main.out -lpthread
$ ./main.out
Hello from first thread!
```

Поток, который можно присоединить, называют соединяемым (joinable). Все потоки таковы по умолчанию. Их противоположность, отсоединенные потоки, невозможно присоединить. Вместо того чтобы присоединять новый поток, функция main делает его отсоединенным. Благодаря этому процесс завершится только после окончания второго потока, несмотря на то что главный поток уже закончился.

Сразу после создания новый поток отсоединяется от главного. Вслед за этим главный поток завершает работу. Инструкция pthread_exit(NULL) нужна для того, чтобы процесс дождался завершения другого, отсоединенного потока. Отсоединенное состояние — один из атрибутов потока, который можно установить перед его созданием и который позволяет его отсоединить.

Пример состояния гонки

Пример, подающий на вывод три разные строки, *пример 30_3*

```
void *thread_body(void *arg) {
    char *str = (char *)arg;
    printf("%s\n", str);
    return NULL;
}

int main(int argc, char **argv) {
    ...
    // Создаем новые потоки
    int result1 = pthread_create(&thread1, NULL, thread_body, "Apple");
    int result2 = pthread_create(&thread2, NULL, thread_body, "Orange");
    int result3 = pthread_create(&thread3, NULL, thread_body, "Lemon");
    ...
    // Ждем, пока потоки не завершат работу
    result1 = pthread_join(thread1, NULL);
    result2 = pthread_join(thread2, NULL);
    result3 = pthread_join(thread3, NULL);
    ...
    return 0;
}
```

```
$ gcc main.c -o
main.out -lpthread
$ ./main.out
Apple
Orange
Lemon
$ ./main.out
Apple
Orange
Lemon
```

Приведенный выше код не является потокобезопасным. Это важный термин; многопоточную программу можно считать потокобезопасной, исключительно когда согласно имеющимся инвариантным ограничениям у нее нет состояния гонки. У нашего кода оно есть, поэтому он не потокобезопасный.

Пример состояния гонки

Пример, в котором литералы выделяются из стека главного потока, *пример 30_4*

```
int main(int argc, char **argv) {  
    ...  
    char str1[8], str2[8], str3[8];  
    strcpy(str1, "Apple");  
    strcpy(str2, "Orange");  
    strcpy(str3, "Lemon");  
    // Создаем новые потоки  
    int result1 = pthread_create(&thread1, NULL, thread_body, str1);  
    int result2 = pthread_create(&thread2, NULL, thread_body, str2);  
    int result3 = pthread_create(&thread3, NULL, thread_body, str3);  
    ...  
    result1 = pthread_detach(thread1); // Отсоединяем потоки  
    result2 = pthread_detach(thread2);  
    result3 = pthread_detach(thread3);  
    ...  
    pthread_exit(NULL); // Теперь строки становятся освобожденными  
    return 0;  
}
```

```
$ gcc main.c -o  
main.out -lpthread  
$ ./main.out  
Apple  
Orange  
Lemon  
$ ./main.out  
Orange  
Apple  
Lemon
```

Следует отметить, что указатель, который передается потоку, не должен быть висячим, иначе могут возникнуть серьезные проблемы с памятью, которые будет сложно отследить.

Пример состояния гонки

Пример 30_4, выполнение примера с помощью профилировщика valgrind

```
$ valgrind ./main.out
..
==3700== Thread 3:
==3700== Conditional jump or move depends on uninitialised value(s)
..
Orange
==3700== Thread 2:
==3700== Conditional jump or move depends on uninitialised value(s)
..
Apple
Lemon
==3700==
==3700== HEAP SUMMARY:
==3700==    in use at exit: 272 bytes in 1 blocks
==3700== total heap usage: 11 allocs, 10 frees, 5,942 bytes allocated
==3700==
==3700== LEAK SUMMARY:
==3700==    definitely lost: 0 bytes in 0 blocks
==3700==    indirectly lost: 0 bytes in 0 blocks
==3700==    possibly lost: 272 bytes in 1 blocks
==3700==    still reachable: 0 bytes in 0 blocks
==3700==    suppressed: 0 bytes in 0 blocks
==3700== Rerun with --leak-check=full to see details of leaked memory
==3700==
==3700== Use --track-origins=yes to see where uninitialised values come from
==3700== For lists of detected and suppressed errors, rerun with: -s
==3700== ERROR SUMMARY: 38 errors from 5 contexts (suppressed: 0 from 0)
```

возможность доступа к области памяти через висячий указатель вовсе не означает, что нам позволено работать с этой областью. Вот почему мы должны использовать профилировщик valgrind, который сообщит нам о таком некорректном доступе к памяти.

Пример гонки данных 1/2

Пример с двумя потоками, которые работают с одной и той же разделяемой переменной, *пример 30_5*

```
void *thread_body_1(void *arg) {
    // Получаем указатель на разделяемую переменную
    int *shared_var_ptr = (int *)arg;
    // Инкрементируем разделяемую переменную на 1, выполняя запись
    // непосредственно по ее адресу в памяти
    (*shared_var_ptr)++;
    printf("%d\n", *shared_var_ptr);
    return NULL;
}

void *thread_body_2(void *arg) {
    // Получаем указатель на разделяемую переменную
    int *shared_var_ptr = (int *)arg;
    // Инкрементируем разделяемую переменную на 2, выполняя запись
    // непосредственно по ее адресу в памяти
    *shared_var_ptr += 2;
    printf("%d\n", *shared_var_ptr);
    return NULL;
}
```

Инвариантное ограничение в данном примере состоит в обеспечении целостности разделяемого состояния; прибавьте к этому все остальные очевидные ограничения, такие как отсутствие сбоев, некорректного доступа к памяти и т. д. Иными словами, способ вывода данных неважен. Главное, чтобы в ситуации, когда один поток изменил значение разделяемой переменной, другой поток, которому это значение неизвестно, не стал его изменять. Вот что мы имеем в виду под целостностью данных.

Пример гонки данных 2/2

Пример с двумя потоками, которые работают с одной и той же разделяемой переменной, **пример 30_5**

```
int main(int argc, char **argv) {  
    ...  
    // Создаем новые потоки  
    int result1 = pthread_create(&thread1, NULL, thread_body_1, &shared_var);  
    int result2 = pthread_create(&thread2, NULL, thread_body_2, &shared_var);  
    ...  
    // Ждем, пока потоки не завершат работу  
    result1 = pthread_join(thread1, NULL);  
    result2 = pthread_join(thread2, NULL);  
    ...  
    return 0;  
}
```

```
$ gcc main.c -o  
main.out -lpthread  
$ ./main.out  
1  
3  
...  
$ ./main.out  
3  
1  
...  
$ ./main.out  
1  
2
```

В данном примере нет локальной переменной для хранения копии разделяемого значения в каждом потоке. Однако вы должны с осторожностью подходить к операциям инкрементирования, поскольку они не атомарны и, следовательно, могут испытывать разные варианты чередований. Каждый поток может менять значение разделяемой переменной с помощью указателя, который он принимает внутри своей функции-компаньона через аргумент `arg`. В обоих вызовах `pthread_create` видно, что четвертым аргументом передается переменная `shared_var`.

Это явный случай гонки данных, а именно по `shared_var`.

На это может потребоваться 100 прогонов!

Адженда

Конкурентность

30 минут

**Синхронизация
потоков**

30 минут

Синхронизация

30 минут

Процессы

30 минут

**Многопоточное
выполнение**

30 минут

**Синхронизация
процессов**

30 минут

Управление конкурентностью в POSIX

POSIX-мьютексы

Использование POSIX-мьютекса для решения проблемы с гонкой данных, пример 30 6

```
pthread_mutex_t mtx;
void *thread_body_1(void *arg) {
    // Получаем указатель на разделяемую переменную
    int *shared_var_ptr = (int *)arg;
    pthread_mutex_lock(&mtx); // Критический участок
    (*shared_var_ptr)++;
    printf("%d\n", *shared_var_ptr);
    pthread_mutex_unlock(&mtx);
    return NULL;
}
void *thread_body_2(void *arg) {
    int *shared_var_ptr = (int *)arg;
    pthread_mutex_lock(&mtx); // Критический участок
    *shared_var_ptr += 2;
    printf("%d\n", *shared_var_ptr);
    pthread_mutex_unlock(&mtx);
    return NULL;
}
int main(int argc, char **argv) {
    ...
    pthread_mutex_init(&mtx, NULL);
    ...
    pthread_mutex_destroy(&mtx);
    ...
}
```

Если скомпилировать этот код и выполнить его какое угодно количество раз, то вывод всегда будет либо 1 3, либо 2 3.

В начале файла мы объявили глобальный объект POSIX-мьютекса, `mtx`.

Первая пара выражений `pthread_mutex_lock(&mtx)` и `pthread_mutex_unlock(&mtx)` в функции-компаньоне `thread_body_1` составляет критический участок первого потока.

Заходя на критический участок, поток блокирует мьютекс, и остальные потоки должны остановиться перед выражением `pthread_mutex_lock(&mtx)` и подождать, пока мьютекс не будет разблокирован снова.

Если вместо этого нам хочется использовать **холостой цикл**? В таком случае пригодится циклическая блокировка. Типы и функции:

- `pthread_spin_t` — тип, используемый для создания объекта циклической блокировки. Аналог `pthread_mutex_t`;
- `pthread_spin_init` — инициализирует объект циклической блокировки. Аналог `pthread_mutex_init`;
- `pthread_spin_destroy` аналог `pthread_mutex_destroy`;
- `pthread_spin_lock` — аналог `pthread_mutex_lock`;
- `pthread_spin_unlock` — аналог `pthread_mutex_unlock`.

Условные переменные POSIX 1/2

Использование условной переменной POSIX для обеспечения определенного порядка выполнения двух потоков, пример 30_7

```
#define TRUE 1
#define FALSE 0
typedef unsigned int bool_t;
// Структура для хранения всех переменных, относящихся
// к разделяемому состоянию
typedef struct {
    // Флаг, показывающий, был выведен символ 'A' или нет
    bool_t done;
    // Объект мьютекса для защиты критического участка
    pthread_mutex_t mtx;
    // Условная переменная для синхронизации двух потоков
    pthread_cond_t cv;
} shared_state_t;
// Инициализирует члены объекта shared_state_t
void shared_state_init(shared_state_t *shared_state) {
    shared_state->done = FALSE;
    pthread_mutex_init(&shared_state->mtx, NULL);
    pthread_cond_init(&shared_state->cv, NULL);
}
// Уничтожает члены объекта shared_state_t
void shared_state_destroy(shared_state_t *shared_state) {
    pthread_mutex_destroy(&shared_state->mtx);
    pthread_cond_destroy(&shared_state->cv);
}
```

Наше инвариантное ограничение состоит в том, чтобы увидеть в выводе сначала A, а затем B (плюс целостность всех разделяемых состояний, отсутствие некорректного доступа к памяти, висячих указателей, сбоев и другие очевидные ограничения).

Здесь используется удобная структура, в которую инкапсулируются разделяемый мьютекс, разделяемая условная переменная и разделяемый флаг. Заметьте, что в каждый поток можно передать только по одному указателю. Поэтому нам пришлось поместить все наши разделяемые ресурсы в одну структурную переменную.

После типов мы определили две функции, предназначенные для инициализации и удаления экземпляров shared_state_t. Их можно считать конструктором и деструктором типа shared_state_t соответственно.

Условные переменные POSIX 2/2

Использование условной переменной POSIX для обеспечения определенного порядка выполнения двух потоков, пример 30_7

```
void *thread_body_1(void *arg) {
    shared_state_t *ss = (shared_state_t *)arg;
    pthread_mutex_lock(&ss->mtx);
    printf("A\n");
    ss->done = TRUE;
    // Шлем сигнал потокам, ожидающим условной переменной
    pthread_cond_signal(&ss->cv);
    pthread_mutex_unlock(&ss->mtx);
    return NULL;
}

void *thread_body_2(void *arg) {
    shared_state_t *ss = (shared_state_t *)arg;
    pthread_mutex_lock(&ss->mtx);
    // Ждем, пока флаг не станет равным TRUE
    while (!ss->done)
        // Ждем условную переменную
        pthread_cond_wait(&ss->cv, &ss->mtx);
    printf("B\n");
    pthread_mutex_unlock(&ss->mtx);
    return NULL;
}
```

Использование условной переменной:

Поток, который ее ждет (пребывая в состоянии «сна»), рано или поздно получает уведомление и «просыпается». Более того, поток может уведомлять (или «пробуждать») любые другие потоки, ожидающие (в состоянии «сна») условную переменную. Все эти операции должны быть защищены мьютексом, и именно поэтому мьютексы всегда необходимо применять в сочетании с условными переменными.

Обязательно обратите внимание на то, что во время ожидания условной переменной связанный с ней мьютекс освобождается и другой поток может продолжить работу. Вдобавок при активации и выходе из состояния ожидания необходимо снова получить мьютекс.

Функцию `pthread_cond_signal` можно использовать для уведомления только единственного потока. флага `done` цикл `while`, а не обычное выражение `if` так как второй поток могут уведомить другие источники, а не только первый поток.

POSIX-барьеры

Решение для примера 30_7 с применением
POSIX-барьеров, пример 30_8

```
pthread_barrier_t barrier;
void *thread_body_1(void *arg) {
    printf("A\n");
    // Ждем присоединения другого потока
    pthread_barrier_wait(&barrier);
    return NULL;
}
void *thread_body_2(void *arg) {
    // Ждем присоединения другого потока
    pthread_barrier_wait(&barrier);
    printf("B\n");
    return NULL;
}
int main(int argc, char **argv) {
    // Инициализируем объект барьера
    pthread_barrier_init(&barrier, NULL, 2);
    ...
    // Уничтожаем объект барьера
    pthread_barrier_destroy(&barrier);
    return 0;
}
```

POSIX-барьеры используют другой метод синхронизации разных потоков.

Мы можем предусмотреть контрольную точку (или точку встречи), при достижении которой все потоки должны остановиться и подождать, пока к ним не присоединятся остальные. Эти контрольные точки можно имитировать с помощью POSIX-барьеров.

У нас было два потока. Один из них должен был выводить А, а другой — В, и мы хотели, чтобы, независимо от чередований, буква А всегда выводилась первой.

Вначале мы объявили глобальный объект барьера с типом `pthread_barrier_t`. Затем внутри функции `main` инициализировали этот объект с помощью функции `pthread_barrier_init`.

Первый аргумент — указатель на объект барьера. **Второй аргумент** — пользовательские атрибуты данного объекта. Мы передаем `NULL`, поэтому атрибуты в объекте барьера будут инициализированы с помощью значений по умолчанию.

Третий аргумент: это количество потоков, которые должны инициировать ожидание барьера путем вызова функции `pthread_barrier_wait`, и их выполнение продолжится только после их освобождения. POSIX-совместимые операционные системы не предоставляют такого механизма, как барьер, в своих интерфейсах системных вызовов, и большинство реализаций основаны на мьютексах и условных переменных.

POSIX-семафоры

В большинстве случаев мьютексов (или двоичных семафоров) достаточно для синхронизации разных потоков, обращающихся к разделяемому ресурсу. Дело в том, что для последовательного выполнения операций чтения и записи на критическом участке должен находиться только один поток. Это называется взаимным исключением (mutual exclusion — отсюда и название «мьютекс»).

Но иногда у вас может возникнуть необходимость в том, чтобы сразу несколько потоков могло работать с разделяемым ресурсом на критическом участке. В таких случаях следует использовать семафоры общего вида.



Двоичные семафоры 1/2

Решение с использованием POSIX-семафоров, пример 30_9

```
#include <pthread.h>
#include <semaphore.h>
sem_t *semaphore;
void *thread_body_1(void *arg) {
    // Получаем указатель на разделяемую переменную
    int *shared_var_ptr = (int *)arg;
    sem_wait(semaphore);    // Ждем семафор
    // Инкрементируем разделяемую переменную на 1,
    // выполняя запись непосредственно по ее адресу в памяти
    (*shared_var_ptr)++;
    printf("%d\n", *shared_var_ptr);
    sem_post(semaphore);    // Освобождаем семафор
    return NULL;
}
void *thread_body_2(void *arg) {
    int *shared_var_ptr = (int *)arg;
    sem_wait(semaphore);
    (*shared_var_ptr) += 2;
    printf("%d\n", *shared_var_ptr);
    sem_post(semaphore);
    return NULL;
}
```

Наша задача состоит в том, чтобы обеспечить целостность разделяемой переменной. **Обратите внимание:** в коде мы не будем использовать POSIX-мьютексы.

Поддержка семафоров появилась в библиотеке потоков POSIX в виде расширения, поэтому они недоступны в заголовке pthread.h. Заглянем внутрь потоков. В функции-компаньоне критические участки защищены sem_wait и sem_post — аналогами функций pthread_mutex_lock и pthread_mutex_unlock соответственно в API для работы с POSIX-мьютексами.

У каждого объекта семафора есть целочисленное значение. Если оно больше нуля и поток, который ожидает семафор, вызывает функцию sem_wait, то значение уменьшается на 1, а потоку разрешается зайти на критический участок. Если оно равно нулю, то поток должен подождать, пока оно снова не станет положительным.

Двоичные семафоры 2/2

Решение с использованием POSIX-семафоров, пример 30_9

```
int main(int argc, char **argv) {
    // Разделяемая переменная
    int shared_var = 0;
    ...
#ifdef __APPLE__
    // Неименованные семафоры не поддерживаются в OS/X. Поэтому семафор нужно
    // инициализировать как именованный, используя функцию sem_open
    semaphore = sem_open("sem0", O_CREAT | O_EXCL, 0644, 1);
#else
    sem_t local_semaphore;
    semaphore = &local_semaphore;
    // Инициализируем семафор как мьютекс (двоичный семафор)
    sem_init(semaphore, 0, 1);
#endif
    // Создаем новые потоки
    int result1 = pthread_create(&thread1, NULL, thread_body_1, &shared_var);
    int result2 = pthread_create(&thread2, NULL, thread_body_2, &shared_var);
    ...
#ifdef __APPLE__
    sem_close(semaphore);
#else
    sem_destroy(semaphore);
#endif
    return 0;
}
```

В системах Apple функции, необходимые для создания неименованных семафоров, помечены как устаревшие, и объект семафора нельзя инициализировать с помощью `sem_init`. Таким образом, мы используем вместо этого функции `sem_open` и `sem_close`, чтобы определить именованные семафоры. Максимальное количество потоков, которым позволено находиться на критическом участке, определяется во время инициализации объекта семафора с помощью последнего аргумента функций `sem_open` и `sem_init`. Мы передали значение 1, поэтому наш семафор должен вести себя подобно мьютексу.

Семафоры общего вида

```
// Барьер для синхронизации потоков водорода и кислорода
pthread_barrier_t water_barrier;
// Мьютекс для синхронизации потоков кислорода
pthread_mutex_t oxygen_mutex;
// Общий семафор для синхронизации потоков водорода
sem_t *hydrogen_sem;
// Разделяемое целое число для подсчета созданных молекул воды
unsigned int num_of_water_molecules;
void *hydrogen_thread_body(void *arg) {
    // На этот критический участок может зайти два потока водорода
    sem_wait(hydrogen_sem);
    // Ждем присоединения другого потока водорода
    pthread_barrier_wait(&water_barrier);
    sem_post(hydrogen_sem);
    return NULL;
}
void *oxygen_thread_body(void *arg) {
    pthread_mutex_lock(&oxygen_mutex);
    // Ждем присоединения потоков водорода
    pthread_barrier_wait(&water_barrier);
    num_of_water_molecules++;
    pthread_mutex_unlock(&oxygen_mutex);
    return NULL;
}
```

Использование семафоров общего вида для имитации процесса создания 50 молекул воды из 50 атомов кислорода и 100 атомов водорода , пример 30_10

Когда количество потоков, ожидающих перед объектом барьера, достигает двух, это означает, что у нас имеется один атом кислорода и два атома водорода; вуаля — мы получаем молекулу воды, и все ожидающие потоки могут продолжить работу. Потоки водорода сразу же завершаются, а поток кислорода сначала инкрементирует счетчик молекул воды.

POSIX-потоки и память

Структура памяти процесса
состоит из нескольких
сегментов, таких как сегменты
кода, стека, данных и кучи.

Сегмент стека

У каждого потока есть собственный сегмент стека, который должен быть доступен только ему. **Стек потока** — часть общего стека процесса-владельца, в котором по умолчанию должны выделяться сегменты стека для всех потоков. Но стек может быть выделен и в куче.

Поскольку все потоки внутри одного процесса могут читать и изменять его сегмент стека, они фактически имеют доступ к стекам друг друга. Но не должны пользоваться этим доступом.

Стоит отметить: работа со стеками других потоков считается опасным поведением, так как переменные, определенные на вершине стека, могут быть уничтожены в любой момент, особенно при завершении потока или возвращении функции.

В однопоточных приложениях есть всего один, главный поток. Поэтому мы используем его сегмент стека так, словно это стек процесса. Но в многопоточных программах все иначе. У каждого потока есть свой стек, который отличается от стеков других потоков.

При создании нового потока в стеке процесса выделяется блок памяти. Если программист самостоятельно не указал его размер, то будет использовано значение по умолчанию, которое зависит от платформы и варьируется от одной архитектуры к другой. В моем случае 8Mb

```
$ ulimit -s  
8192
```



Сегмент стека

Назначение блока кучи в качестве стека потока, пример 30_11

API для работы с POSIX-потоками позволяет назначить новому потоку область стека. В примере у нас два потока. В одном из них мы используем параметры стека по умолчанию, а в другом выделяем буфер в сегменте кучи и назначаем его в качестве стека. Стоит отметить, что для выделяемого буфера следует указывать минимальный размер, иначе его нельзя будет использовать в качестве стека.

\$./main.out

Thread1 > Stack Address:

0x7f3a0320ae44

Thread2 > Stack Address:

0x55c3f0b1d144

Main Thread > Heap Address:

0x55c3f0b1a2a0

Main Thread > Stack Address:

0x7ffedb333c70

```
void *thread_body_1(void *arg) {
    int local_var = 0;
    printf("Thread1 > Stack Address: %p\n", (void *)&local_var);
    return 0;
}

...
int main(int argc, char **argv) {
    size_t buffer_len = PTHREAD_STACK_MIN + 100;
    // Буфер, выделенный из кучи и используемый как стек потока
    char *buffer = (char *)malloc(buffer_len * sizeof(char));
    pthread_t thread1; // Обработчики потоков
    pthread_t thread2;
    // Создаем новый поток с атрибутами по умолчанию
    int result1 = pthread_create(&thread1, NULL, thread_body_1, NULL);
    // Создаем новый поток с нашим собственным стеком
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    // Задаем адрес и размер стека
    if (pthread_attr_setstack(&attr, buffer, buffer_len)) {
        printf("Failed while setting the stack attributes.\n");
        exit(1);
    }
    int result2 = pthread_create(&thread2, &attr, thread_body_2, NULL);
    ...
    printf("Main Thread > Heap Address: %p\n", (void *)buffer);
    printf("Main Thread > Stack Address: %p\n", (void *)&buffer_len);
    ...
}
```

Сегмент стека

Попытка прочитать переменную, выделенную в области стека другого потока, пример 30_12

```
int *shared_int;

void *t1_body(void *arg) {
    int local_var = 100;
    shared_int = &local_var;
    // Ждем, пока другой поток не выведет разделяемое целое значение
    usleep(10);
    return NULL;
}

void *t2_body(void *arg) {
    printf("%d\n", *shared_int);
    return NULL;
}

int main(int argc, char **argv) {
    shared_int = NULL;
    pthread_t t1;
    pthread_t t2;
    pthread_create(&t1, NULL, t1_body, NULL);
    pthread_create(&t2, NULL, t2_body, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Вначале мы объявили глобальный разделяемый указатель.

В представленном коде внутри функции-компаньона `t1_body` мы сохраняем адрес локальной переменной в разделяемый указатель. Эта переменная принадлежит первому потоку и выделена на вершине его стека.

С этого момента по завершении первого потока разделяемый указатель станет висячим, и любое его разыменовывание, скорее всего, приведет к сбою, логической ошибке или в лучшем случае к неочевидным проблемам с памятью.

```
$ gcc main.c -o main.out -lpthread
$ ./main.out
100
$ ./main.out
100
$ ./main.out
100
$ ./main.out
0
```


Сегмент кучи

Сегменты кучи и данных доступны всем потокам, однако первый является динамическим и разделяется во время выполнения, тогда как второй генерируется на этапе компиляции. Потоки могут читать и изменять содержимое кучи; оно существует на протяжении всего времени жизни процесса и не зависит от отдельных его потоков. Кроме того, в кучу можно поместить большие объекты. Благодаря совокупности всех этих факторов **куча** — отличное место для хранения состояний, которые должны разделяться между разными потоками.

Когда дело доходит до выделения кучи, управление памятью превращается в настоящий кошмар. Причиной тому факт, что выделяемые ресурсы в конечном счете должен освободить один из активных потоков, иначе можно столкнуться с утечками памяти.

Применительно к конкурентным средам чередования могут легко привести к появлению висячих указателей, в результате чего возникают сбои. Важнейшая цель синхронизации состоит в упорядочении выполнения таким образом, чтобы висячие указатели не могли появиться, и достичь этого непросто.



Сегмент кучи

Пример без каких-либо механизмов синхронизации, пример 30_13

```
char *shared_array; // Указатель на разделяемый массив
unsigned int shared_array_len; // Размер разделяемого массива
void *alloc_thread_body(void *arg) {
    shared_array_len = 20;
    shared_array = (char *)malloc(shared_array_len * sizeof(char *));
    return NULL;
}
void *filler_thread_body(void *arg) {
    int even = *((int *)arg);
    char c = 'a';
    size_t start_index = 1;
    if (even) {
        c = 'Z';
        start_index = 0;
    }
    for (size_t i = start_index; i < shared_array_len; i += 2)
        shared_array[i] = even ? c-- : c++;
    shared_array[shared_array_len - 1] = '\0';
    return NULL;
}
void *printer_thread_body(void *arg) {
    printf(">> %s\n", shared_array);
    return NULL;
}
void *dealloc_thread_body(void *arg) {
    free(shared_array);
    return NULL;
}
```

Пример, представленный ниже, состоит из пяти потоков. Первый выделяет массив в куче, а второй и третий заполняют его следующим образом. Второй присваивает элементам с четными индексами прописные буквы, начиная с Z и двигаясь обратно к A; третий присваивает элементам с нечетными индексами строчные буквы, начиная с a и двигаясь вперед к z. Четвертый поток выводит полученный массив, а пятый его удаляет и освобождает память в куче.

В данном примере следует применить все методы управления конкурентностью с помощью средств POSIX, чтобы не позволить этим потокам выйти из-под контроля при работе с кучей. Пример не содержит управляющего механизма и потому явно не является типобезопасным.

Сегмент кучи

Пример с механизмами синхронизации, пример 30_14

```

...
void *alloc_thread_body(void *arg) {
    shared_array_len = 20;
    shared_array = (char *)malloc(shared_array_len * sizeof(char *));
    pthread_barrier_wait(&alloc_barrier);
    return NULL;
}

...
void *dealloc_thread_body(void *arg) {
    pthread_barrier_wait(&done_barrier);
    free(shared_array);
    pthread_barrier_destroy(&alloc_barrier);
    pthread_barrier_destroy(&fill_barrier);
    pthread_barrier_destroy(&done_barrier);
    return NULL;
}

int main(int argc, char **argv) {
    shared_array = NULL;
    pthread_barrier_init(&alloc_barrier, NULL, 3);
    ...
    int res = pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
    CHECK_RESULT(res);
    res = pthread_create(&alloc_thread, &attr, alloc_thread_body, NULL);
    ...
    pthread_exit(NULL);
    return 0;
}

```

Чтобы сделать код в данном листинге потокобезопасным, достаточно добавить в него POSIX-барьеры. Это самый простой способ гарантировать последовательное выполнения нескольких потоков. Самое простое и зачастую оптимальное место для хранения разделяемых состояний (с точки зрения усилий, которые программист должен приложить, чтобы управлять памятью) — сегмент данных, в котором операции выделения и освобождения выполняются автоматически. Переменные, находящиеся в этом сегменте, считаются глобальными и имеют максимально продолжительное время жизни — с момента создания процесса до самого его завершения.

Видимость памяти

Протокол когерентности кэша ядер процессора следит за тем, чтобы все закэшированные версии одного адреса памяти во всех ядрах оставались синхронизированными и соответствовали последним изменениям, внесенным в одно из ядер. Однако этот протокол должен как-то инициироваться.

Существуют системные вызовы, которые приводят к срабатыванию протокола когерентности кэша и делают память видимой для всех ядер процессора. В библиотеке pthread тоже есть ряд функций, перед выполнением которых гарантируется видимость памяти.

Помимо локальных кэшей в процессорных ядрах, компиляторы могут применять механизмы кэширования для часто используемых переменных. Для этого компилятор должен проанализировать код и оптимизировать его таким образом, чтобы часто используемые переменные записывались и считывались из его кэша. Переменные, которые компилятор не должен оптимизировать с помощью кэширования, можно объявлять

изменчивыми.

Изменчивая переменная объявляется с помощью ключевого слова `volatile`. пример объявления целочисленной изменчивой переменной.

Объявление целочисленной изменчивой переменной
`volatile int number;`

СИСТЕМНЫЕ ВЫЗОВЫ

- `pthread_barrier_wait;`
- `pthread_cond_broadcast;`
- `pthread_cond_signal;`
- `pthread_cond_timedwait;`
- `pthread_cond_wait;`
- `pthread_create;`
- `pthread_join;`
- `pthread_mutex_lock;`
- `pthread_mutex_timedlock;`
- `pthread_mutex_trylock;`
- `pthread_mutex_unlock;`
- `pthread_spin_lock;`
- `pthread_spin_trylock;`
- `pthread_spin_unlock;`
- `pthread_rwlock_rdlock;`
- `pthread_rwlock_timedrdlock;`
- `pthread_rwlock_timedwrlock;`
- `pthread_rwlock_tryrdlock;`
- `pthread_rwlock_trywrlock;`
- `pthread_rwlock_unlock;`
- `pthread_rwlock_wrlock;`
- `sem_post;`
- `sem_timedwait;`
- `sem_trywait;`
- `sem_wait;`
- `semctl;`
- `semop.`

Адженда

Конкурентность

30 минут

**Синхронизация
потоков**

30 минут

Синхронизация

30 минут

Процессы

30 минут

**Многопоточное
выполнение**

30 минут

**Синхронизация
процессов**

30 минут

API для выполнения процессов

Любая программа выполняется в виде процесса. Но поначалу она представляет собой исполняемый двоичный файл, содержащий некоторые сегменты памяти и, скорее всего, множество инструкций машинного уровня. А вот процесс — это отдельный экземпляр программы на этапе ее выполнения. Таким образом, одну скомпилированную программу (или исполняемый двоичный файл) можно выполнять по нескольку раз в разных процессах.

Стоит отметить, что нас в основном интересует выполнение процессов в Unix-подобных операционных системах, поскольку все они имеют многослойную архитектуру, свойственную Unix, и предоставляют широко известные и похожие API. У других ОС могут быть собственные механизмы для выполнения процессов, но большинство из них более или менее соответствуют многослойной архитектуре, поэтому от них можно ожидать наличия аналогичных методов.

В Unix-подобной операционной системе есть не так уж много способов выполнить процесс на уровне системных вызовов. Кольцо ядра находится почти в самом центре, сразу после кольца аппаратного обеспечения, и предоставляет внешним кольцам (командной оболочке и прикладным программам) различные возможности ядра в виде интерфейса системных вызовов. Два таких системных вызова предназначены для создания и выполнения процессов; речь идет о `fork` и `exec` (или `execve` в Linux) соответственно. Создание подразумевает порождение нового процесса, а при выполнении мы берем имеющийся процесс и подставляем в него новую программу; таким образом, во втором случае новый процесс не порождается.



API для выполнения процессов

В результате использования этих системных вызовов программа всегда выполняется в виде нового процесса, но данный процесс порождается не всегда! Вызов `fork` порождает новый процесс, а вызов `exec` заменяет вызывающий процесс новым.

Unix-подобные операционные системы имеют два стандарта, которые среди прочего описывают интерфейс, предоставляемый кольцу командной оболочки. Эти стандарты — единая спецификация Unix (Single Unix Specification, SUS) и POSIX. Интерфейс, предоставляемый кольцом ядра, должен быть подробно описан в стандарте POSIX, и, действительно, некоторые аспекты этого стандарта посвящены выполнению и управлению процессами.

Поэтому, как следовало ожидать, в POSIX должны присутствовать заголовки и функции для создания и выполнения процессов. И они там действительно есть; соответствующие возможности обнаруживаются в разных заголовочных файлах. POSIX-функции, предусмотренные для создания и выполнения процессов, перечислены ниже:

- функция `fork`, которая находится в заголовочном файле `unistd.h`, отвечает за создание процесса;
- функции `posix_spawn` и `posix_spawnnp`, находящиеся в заголовочном файле `spawn.h`, отвечают за создание процесса;
- функции семейства `exec*`, такие как `exec` и `execp`, находятся в заголовочном файле `unistd.h` и отвечают за выполнение процесса.



API для выполнения процессов

Детали реализации функции `fork` в Cygwin можно найти [по адресу](#). Стоит отметить, что она не использует внутри системный вызов `fork`, который обычно присутствует в Unix-подобных ядрах; вместо этого она подключает заголовки из Win32 API и вызывает общеизвестные функции, предназначенные для создания процессов и управления ими.

Согласно спецификации POSIX, кольцо командной оболочки в Unix-подобных системах предоставляет не только стандартную библиотеку C. В терминале есть предустановленные командные утилиты, которые позволяют использовать стандартные функции C в сложных сценариях. И каждый раз, когда пользователь вводит команду в терминале, создается новый процесс. Даже такие простые команды, как `ls` или `sed`, порождают новые процессы, которые могут проработать меньше одной секунды.

В монолитных системах, таких как Linux или FreeBSD, механизм отслеживания заданий (процессов и потоков) находится внутри ядра, поэтому логично, что процессы создаются в самом ядре.

Создание новых процессов требует родительского процесса. Вот почему у любого процесса есть родитель. На самом деле он может быть только один. Цепочка родителей и прародителей растягивается до самого первого пользовательского процесса, который обычно называется `init`. Родителем `init` служит процесс ядра.

Такие отношения между родителями и потомками формируют большое дерево процессов, которое можно просмотреть с помощью командной утилиты `ps tree`.



Создание процесса

Создание дочернего процесса с помощью API fork, пример 30_15

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char **argv)
{
    printf("This is the parent process with process ID: %d\n",
           getpid());
    printf("Before calling fork() ...\n");
    pid_t ret = fork();
    if (ret)
        printf("The child process is spawned with PID: %d\n", ret);
    else
        printf("This is the child process with PID: %d\n", getpid());
    printf("Type CTRL+C to exit ...\n");
    while (1)
        ;
    return 0;
}
```

```
$ ./main.out
This is the parent process with process ID: 668
Before calling fork() ...
The child process is spawned with PID: 669
Type CTRL+C to exit ...
This is the child process with PID: 669
Type CTRL+C to exit ...
^C
$
```

Чтобы создать новый дочерний процесс, родительскому процессу необходимо вызвать функцию `fork`. Ее объявление можно подключить вместе с заголовочным файлом `unistd.h`, который входит в состав POSIX.

При вызове функции `fork` создается точная копия вызывающего (родительского) процесса. В результате оба процесса продолжают работать конкурентно, начиная с инструкции, которая идет за вызовом `fork`. Дочерний процесс наследует много атрибутов от своего родителя, включая все сегменты памяти и их содержимое. Следовательно, он имеет доступ ко всем тем же переменным в сегментах данных и кучи, а также содержит те же программные инструкции в сегменте кода.

Какие наследуются атрибуты?

Создание процесса

Использование утилиты `pstree` для поиска процессов

```
$ pstree -p
systemd(1)─agetty(248)
          └─agetty(250)
          └─cron(201)
          └─dbus-daemon(203)
          └─init-
systemd(Ub(2)─SessionLeader(390)─Relay(392)(391)─bash(392)─pstree(3208)
            │
            └─SessionLeader(1762)─Relay(1768)(1764)─bash(1768)─main.out(3162)+++
                  │
                  └─{init(7)─{init}(8)
                     └─login(393)─bash(435)
                        └─{init-systemd(Ub}(9)
...

```

В последней строчке этого терминала указано два процесса с PID 3162 и 3162, которые имеют отношения вида «родитель — потомок». Интересно, что родитель процесса 3162 имеет PID 1768, который принадлежит процессу `bash`.

В Linux самым первым процессом является планировщик, который входит в образ ядра и имеет PID 0. Следующий процесс обычно называется `init`, и его PID равен 1; это первый процесс, который создается планировщиком.

Однако в более новых версиях почти всех известных дистрибутивов Linux процесс `init` был заменен демоном `systemd`. [Материал об отличиях `init` и `systemd`](#).



Создание процесса

Два процесса, которые записывают разные строки в стандартный вывод, пример 30_16

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char **argv) {
    pid_t ret = fork();
    if (ret)
        for (size_t i = 0; i < 5; i++) {
            printf("AAA\n");
            usleep(1);
        }
    else
        for (size_t i = 0; i < 5; i++) {
            printf("BBBBBB\n");
            usleep(1);
        }
    return 0;
}
```

```
$ gcc main.c -o main.out
$ ./main.out
AAA
BBBBBB
AAA
BBBBBB
AAA
BBBBBB
AAA
BBBBBB
AAA
BBBBBB
$ ./main.out
AAA
BBBBBB
BBBBBB
AAA
AAA
BBBBBB
AAA
BBBBBB
AAA
BBBBBB
```

При использовании функции `fork` родительский и дочерний процессы выполняются конкурентно. Это значит, что они должны проявлять некоторые свойства конкурентных систем. Если определить наше инвариантное ограничение в соответствии с тем, что мы видим в стандартном выводе, то существует риск состояния гонки.

Выполнение процесса

Демонстрация работы функции `execvp`, пример 30_17

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>
int main(int argc, char **argv) {
    char *args[] = {"echo", "Hello", "World!", 0};
    execvp("echo", args);
    printf("execvp() failed. Error: %s\n", strerror(errno));
    return 0;
}
```

Таким образом, использование функций `exec*` вместо создания нового процесса вызывает подмену уже существующего. Это самое важное отличие от функции `fork`. Базовый процесс не копируется, а полностью заменяется новым набором сегментов памяти и программных инструкций. Функция `execvp` входит в семейство функций `exec*`, для запуска процесса `echo`. Функция `execvp` — одна из функций группы `exec*`, которая наследует от родительского процесса переменную среды `PATH` и ищет исполняемые файлы так, как это делал родитель.

Обратите внимание: мы указали `echo`, а не абсолютный путь. Таким образом, функция `execvp` должна сначала найти соответствующий исполняемый файл.

Еще один способ выполнить новый процесс заключается в использовании функций семейства `exec*`. В них применяется другой подход по сравнению с API `fork`. Философию, стоящую за функциями `exec*`, можно описать так: вначале создается простой базовый процесс, и затем в какой-то момент в него загружается нужный нам исполняемый файл, который становится новым образом процесса. Это загруженная версия исполняемого файла с выделенными сегментами памяти, готовая к выполнению.

Выполнение процесса

Функции `exec*` могут запускать разного рода исполняемые файлы. Ниже перечислены все файловые форматы, которые поддерживаются этими функциями:

- исполняемые файлы ELF;
- скриптовые файлы с символами `#!` в самом начале, которые обозначают интерпретатор скрипта;
- традиционный формат двоичных файлов `a.out`;
- исполняемые файлы ELF FDPIC.

Обнаружив исполняемый файл `echo`, функция `execvp` делает все остальное. Она инициирует системный вызов `exec` (`execve` в Linux) с подготовленным набором аргументов, в результате чего ядро формирует из найденного исполняемого файла образ процесса. Когда все готово, ядро заменяет текущий образ процесса новым, и после этого базовый процесс теряется навсегда. Дальше управление переходит к новому процессу, который начинает работу с функции `main`, как при обычном выполнении.



Выполнение процесса

Как уже говорилось ранее, `execvp` — это лишь одна из функций семейства `exec*`. И несмотря на похожее поведение, они все же слегка отличаются. Ниже приводится их сравнение.

- Функция `execl(const char* path, const char* arg0, ..., NULL)` принимает абсолютный путь к исполняемому файлу вместе с набором аргументов, которые должны быть переданы новому процессу. Пример, переписанный с использованием `execl`, выглядел бы так: `execl("/usr/bin/echo", "echo", "Hello", "World", NULL)`.
- Функция `execlp(const char* file, const char* arg0, ..., NULL)` принимает относительный путь в качестве первого аргумента и может легко найти исполняемый файл благодаря доступу к переменной среды `PATH`. Пример, переписанный с использованием `execlp`, выглядел бы так: `execlp("echo", "echo," "Hello," "World," NULL)`.
- Функция `execle(const char* path, const char* arg0, ..., NULL, const char* env0, ..., NULL)` принимает абсолютный путь к исполняемому файлу вместе с набором аргументов, которые должны быть переданы новому процессу. Пример, переписанный с использованием `execle`, выглядел бы так: `execle("/usr/bin/echo", "echo", "Hello", "World", NULL, "A=1", "B=2", NULL)`.
- Функция `execv(const char* path, const char* args[])` принимает абсолютный путь к исполняемому файлу вместе с массивом аргументов, которые должны быть переданы новому процессу. Пример, переписанный с использованием `execl`, выглядел бы так: `execl("/usr/bin/echo", args)`, где переменная `args` была бы объявлена как `char* args[] = {"echo", "Hello", "World", NULL}`.
- Функция `execvp(const char* file, const char* args[])` принимает относительный путь в качестве первого аргумента и может легко найти исполняемый файл благодаря доступу к переменной среды `PATH`. В случае успешного выполнения функций `exec*` предыдущий процесс исчезает, а его место занимает новый.

Разные методы создания и выполнения процессов

Учитывая все вышесказанное и примеры, приведенные выше, мы можем сравнить два метода выполнения новой программы.

- В результате успешного вызова `fork` получаются два отдельных процесса: родительский, который вызывал `fork`, и дочерний. Успешное выполнение `exec*` заключается в замене вызывающего процесса новым образом, в результате чего новый процесс не создается.
- Вызов `fork` дублирует все содержимое памяти родительского процесса, поэтому потомку доступны все те же адреса и переменные. Для сравнения, вызов `exec*` уничтожает структуру памяти базового процесса и создает новую структуру, основанную на загруженном исполняемом файле.
- У скопированного процесса есть доступ к определенным атрибутам родителя, таким как открытые файловые дескрипторы. А вот при использовании функций `exec*` новый процесс не знает об атрибутах старого и ничего от него не наследует.
- В обоих случаях новый процесс содержит всего один главный поток. Поток родительского процесса не копируется при использовании `fork`.
- Функции `exec*` можно применять для выполнения скриптов и внешних исполняемых файлов, а `fork` подходит только для создания новых процессов, которые являются копией той же программы.

Этапы выполнения процесса

Чтобы выполнить процесс из исполняемого файла, в большинстве операционных систем пространства ядра и пользователя проходят через ряд общих этапов. С точки зрения кольца прикладных приложений для этого следует инициировать системный вызов, такой как `exec`.

Обратите внимание: в случае со скриптом загружается программа-интерпретатор, которая обычно указывается в первой строке скрипта вслед за символами `#!`. Чтобы выполнить процесс, загрузчик обязан произвести следующие действия:

- проверить контекст выполнения и права доступа пользователя, который запросил выполнение;
- выделить для нового процесса адресное пространство в основной памяти;
- скопировать двоичное содержимое исполняемого файла в выделенное пространство. Это в основном касается сегментов данных и кода;
- выделить область памяти для сегмента стека и подготовить начальные привязки памяти;
- создать главный поток выполнения и его сегмент стека;
- скопировать аргументы командной строки в стековый фрейм и поместить его на вершину стека главного потока;
- инициализировать регистры, необходимые для выполнения;
- выполнить первую инструкцию точки входа в программу.

В случае со скриптом путь к файлу копируется в качестве аргумента командной строки для процесса интерпретатора. Описанные выше этапы присущи большинству ядер, но детали их реализации могут варьироваться. Выполнение процессов в Linux (на сайте [LWN](http://lwn.net)).

Разделяемые состояния

Процессы, как и потоки, могут разделять между собой состояния. Единственное отличие в том, что потоки имеют доступ к общему пространству памяти, которое принадлежит их процесс-владельцу. Процессы лишены такой роскоши, поэтому для разделения состояний между ними необходимо использовать другие механизмы.



Методы разделения ресурсов

Если проанализировать методы разделения состояния (переменной или массива) между двумя процессами, окажется, что их не так уж много.

Состояние либо размещается в каком-то «месте», доступном ряду процессов, либо отправляется (или передается) другим процессам в виде сообщения, сигнала или события. Точно так же вы должны либо извлечь имеющееся состояние из заданного «места», либо принять его в виде сообщения, сигнала или события. Для первого подхода требуется хранилище (или носитель), такое как буфер памяти или файловая система, а для второго — механизм обмена сообщениями, или канал, между процессами.

В качестве примера первого подхода вполне уместен массив, который находится в разделяемой области памяти и может быть прочитан и изменен разными процессами. Что касается второго подхода, то это может быть компьютерная сеть, служащая каналом для передачи сообщений между процессами, размещенными на разных компьютерах в данной сети.

В другой терминологии методики, относящиеся к первой группе и требующие носитель или хранилище для разделения состояний, называются активными. Это объясняется тем, что процесс, который хочет прочитать состояние, должен извлечь его из хранилища.

Методики из второй группы, которым нужен канал для передачи состояния, называются пассивными, поскольку состояние доставляется принимающему процессу по каналу и тот не должен извлекать его из носителя. Изобилие пассивных методик привело к появлению в современной индустрии разработки ПО различных распределенных архитектур. Активные методики считаются устаревшими по сравнению с пассивными, и вы можете встретить их во многих корпоративных приложениях, в которых для разделения состояния всей системы используется единая центральная база данных.

Методы разделения ресурсов

В настоящее время все более популярным становится пассивный подход. Он привел к появлению таких шаблонов проектирования, как источник событий (event sourcing) и других похожих методик, которые позволяют сохранять все элементы программной системы в согласованном состоянии, не прибегая к централизованному хранению всех данных.

Активные методики, которые поддерживаются в стандарте POSIX и могут свободно использоваться во всех POSIX-совместимых операционных системах.

- **Разделяемая память.** Это просто область главной памяти, которая разделяется между разными процессами и доступна каждому из них; они могут использовать ее как обычный блок памяти для хранения переменных и массивов. *Объект разделяемой памяти* — настоящая оперативная память, а не файл на диске.
- **Файловая система.** Процессы могут разделять состояние с помощью файловой системы. Это одна из старейших методик разделения состояния в программной системе между разными процессами. В какой-то момент трудности синхронизации доступа к общим файлам в сочетании со многими другими справедливыми причинами привели к изобретению систем управления базами данных (СУБД), но данный подход до сих пор используется в определенных ситуациях.
- **Сетевые сервисы.** Процессы могут использовать сетевые хранилища или сервисы для хранения и извлечения разделяемого состояния. В этом случае процессы не знают, что именно происходит внутри. Они просто обращаются к сетевым сервисам через строго определенный API, который позволяет им выполнять различные операции с разделяемым состоянием.

Разделяемая память в POSIX

```
#define SH_SIZE 16
int main(int argc, char **argv) {
    int shm_fd = shm_open("/shm0", O_CREAT | O_RDWR, 0600);
    ...
    fprintf(stdout, "Shared memory is created with fd: %d\n", shm_fd);
    if (ftruncate(shm_fd, SH_SIZE * sizeof(char)) < 0) {
        fprintf(stderr, "ERROR: Truncation failed: %s\n", strerror(errno));
        return 1;
    }
    fprintf(stdout, "The memory region is truncated.\n");
    void *map = mmap(0, SH_SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (map == MAP_FAILED) {
        fprintf(stderr, "ERROR: Mapping failed: %s\n",
            strerror(errno));
        return 1;
    }
    char *ptr = (char *)map;
    ptr[0] = 'A';
    ptr[1] = 'B';
    ptr[2] = 'C';
    ptr[3] = '\n';
    ptr[4] = '\0';
    ...
    return 0;
}
```

```
$ ls /dev/shm
$ ./main.out
Shared memory is created with fd: 3
The memory region is truncated.
^C
$ ls /dev/shm
shm0
$ cat /dev/shm/shm0
ABC
```

Разделяемая память поддерживается стандартом POSIX и является одним из самых распространенных методов разделения информации между разными процессами. Процессы, в отличие от потоков, не имеют доступа к общему пространству памяти; он запрещен на уровне операционной системы. Следовательно, нам нужен какой-то механизм, который позволит разделять участки памяти между двумя процессами, и разделяемая память — именно такой механизм.

Разделяемая память в POSIX

Данный код создает объект разделяемой памяти с именем /shm0 и 16 байтами внутри. Затем он записывает туда литерал ABC\n и выходит, отвязываясь от разделяемой области памяти.

Обратите внимание: эта область остается на месте даже после завершения программы. Будущие процессы могут снова его открыть и прочитать. Объект разделяемой памяти уничтожается либо во время перезагрузки системы, либо самим процессом.

В FreeBSD имена объектов разделяемой памяти должны начинаться с /.

Функция **shm_open** для открытия объекта разделяемой памяти. Она принимает имя объекта и его режимы. Элементы O_CREAT и O_RDWR означают, что разделяемая память должна быть создана и доступна как для чтения, так и для записи.

С помощью функции **ftruncate** определяем размер разделяемой области. Этот шаг необходим, если вы хотите создать новый объект разделяемой памяти.

Используя функцию **mmap** мы привязали наш объект к области, доступной процессу. В результате получился указатель на привязанную память, который можно применять для обращения к соответствующей области. Это тоже обязательный шаг, который делает разделяемую память доступной для нашей программы на языке C. Элемент PROT_WRITE указывает на то, что область привязывается с возможностью записи, а MAP_SHARED означает ее разделение между процессами.

После привязки области разделяемой памяти приведенный выше код записывает в нее строку ABC\n; обратите внимание на символ перевода строки в конце. Затем процесс отвязывает область разделяемой памяти с помощью функции **munmap** и закрывает файловый дескриптор, назначенный объекту этой области.

Разделяемая память в POSIX

Каждая операционная система предлагает собственный способ создания неименованных или анонимных объектов разделяемой памяти. В FreeBSD для этого достаточно передать функции **shm_open** в качестве пути к объекту разделяемой памяти параметр SHM_ANON. В Linux вместо объекта можно создать анонимный файл, используя функцию **memfd_create**, и затем передать его дескриптор для создания привязанной области. Анонимная разделяемая память принадлежит владеющему ей процессу и не может применяться для разделения состояний между несколькими процессами.

В Linux объекты разделяемой памяти доступны в каталоге /dev/shm.

Обратите внимание: он находится не в обычной файловой системе, поэтому то, что вы в нем видите, не является файлами на диске. Для /dev/shm используется файловая система shmfs. Она доступна только в Linux и предназначена для предоставления доступа к временным объектам внутри памяти через подключенный каталог.

Создание объекта разделяемой памяти подтверждается и выводом программы. Еще один аспект этого терминала, заслуживающий внимания, — дескриптор 3, который назначается объекту разделяемой памяти. В каждом процессе файловые дескрипторы начинаются с 0 и инкрементируются, пока не достигнут максимально разрешенного числа.

Следует отметить, что в каждом процессе номера файловых дескрипторов 0, 1 и 2 заранее назначены потокам данных stdout, stdin и stderr соответственно.

В macOS для просмотра активных IPC-объектов, существующих в системе, можно использовать утилиту **ps**. Она умеет выводить очереди активных сообщений и объекты разделяемой памяти. С ее помощью также можно проверить активные семафоры.

Разделяемая память в POSIX

```

...
#define SH_SIZE 16
int main(int argc, char **argv) {
    int shm_fd = shm_open("/shm0", O_RDONLY, 0600);
    ...
    fprintf(stdout, "Shared memory is opened with fd: %d\n", shm_fd);
    void *map = mmap(0, SH_SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);
    ...
    char *ptr = (char *)map;
    fprintf(stdout, "The contents of shared memory object: %s\n",
            ptr);
    if (munmap(ptr, SH_SIZE) < 0) {
        fprintf(stderr, "ERROR: Unmapping failed: %s\n",
                strerror(errno));
        return 1;
    }
    ...
    return 0;
}

```

```

$ ls /dev/shm
shm0
$ gcc main.c -o main.out
$ ./main.out
Shared memory is opened with fd: 3
The contents of shared memory
object: ABC
$

```

Пример 30_19, чтение из объекта разделяемой памяти, созданного в примере 30_18

Первое выражение в функции main открывает существующий объект разделяемой памяти с именем /shm0. В следующих строках мы привязываем область разделяемой памяти. Опять же, передавая аргумент PROT_READ, мы указываем, что эта область предназначена только для чтения. После этого мы наконец получаем указатель на разделяемую память и используем его для вывода ее содержимого. Закончив, мы отвязываем область. В конце объект разделяемой памяти регистрируется для удаления с помощью функции shm_unlink.

Пример гонки данных с использованием разделяемой памяти

Демонстрация гонки данных при использовании разделяемой памяти POSIX и функции fork, пример 30_20

```
$ gcc main.c -o main.out
$ ./main.out
File is created and opened with fd: 3
The memory region is truncated.
The parent process sees the counter as 1.
The child process sees the counter as 1.
The child process finished with status 0.
$ ./main.out
File is created and opened with fd: 3
The memory region is truncated.
The parent process sees the counter as 2.
The child process sees the counter as 1.
The child process finished with status 0.
```

Когда родительский процесс изменяет разделяемый счетчик, он ждет завершения своего потомка и только затем пытается отвязать, закрыть и удалить объект разделяемой памяти. Следует отметить, что отвязка и закрытие файлового дескриптора происходит в обоих процессах, но удаление выполняет только родитель. Внутри функции `inc_counter` используются необычные вызовы `usleep`. Это делается для того, чтобы заставить планировщик заданий передать ядро процессора от одного процесса к другому. Без вызова `usleep` процессорное ядро обычно не передается между процессами, и потому результаты разных чередований проявлялись бы не так часто.

Родительский процесс создает и открывает объект разделяемой памяти и назначает ему файловый дескриптор до вызова функции `fork`. Как видите, у нас получилась явная гонка данных, относящаяся к разделяемому счетчику.

Файловая система

В стандарте POSIX есть аналогичный API для работы с объектами файловой системы. Он ничем не отличается от того, с помощью которого мы получали доступ к разделяемой памяти. Он позволяет ссылаться на различные системные объекты — главное, чтобы для этого использовались файловые дескрипторы.

Файловые дескрипторы дают возможность ссылаться на файлы, хранящиеся в таких файловых системах, как ext4, а также в разделяемой памяти, каналах и т. д. Более того, вам доступна та же семантика для открытия, чтения, записи и привязки дескрипторов к локальной области памяти. Файловые дескрипторы обычно привязываются (отображаются). Однако существуют определенные исключительные случаи, когда привязать можно дескрипторы сокетов. Дескрипторы сокетов похожи на файловые, но используются для сетевых или Unix-сокетов. [Пример](#) привязки буфера ядра к TCP-сокету, известный как механизм приема данных без копирования.

На самом деле объекты файловой системы, находящиеся на жестком диске, фундаментально отличаются от объектов разделяемой памяти. Проведем краткий обзор этих различий.

- Объект разделяемой памяти — это, в сущности, адресное пространство процесса ядра, а объект файловой системы находится на диске. В крайнем случае файл имеет выделенные буферы для операций чтения и записи.
- Состояния, записанные в разделяемую память, стираются при перезагрузке системы, а состояния, сохраненные в разделяемый файл, остаются на месте, если этот файл находится на жестком диске или на другом постоянном накопителе.
- Доступ к разделяемой памяти обычно быстрее доступа к файловой системе.

Файловая система

Демонстрация гонки данных на примере обычных файлов и функции `fork`, пример 30_21

```
// Разделяемый файловый дескриптор, с помощью которого
// мы ссылаемся на разделяемый файл
int shared_fd = -1;
// Указатель на разделяемый счетчик
int32_t *counter = NULL;
void init_shared_resource() {
    // Открываем файл
    shared_fd = open("data.bin", O_CREAT | O_RDWR, 0600);
    if (shared_fd < 0) {
        fprintf(stderr, "ERROR: Failed to create the file: %s\n",
                strerror(errno));
        exit(1);
    }
    fprintf(stdout, "File is created and opened with fd: %d\n",
            shared_fd);
}
void shutdown_shared_resource() {
    if (remove("data.bin") < 0) {
        fprintf(stderr, "ERROR: Removing the file failed: %s\n",
                strerror(errno));
        exit(1);
    }
}
```

Вместо `shm_open` и `shm_unlink` используются функции `open` и `remove`.

Обратите внимание: файл `data.bin` создается в текущем каталоге, поскольку мы не передавали функции `open` абсолютный путь.

Сравнение многопоточности и многопроцессности

Многопоточность

В первом сценарии мы можем написать приложение, состоящее из одного процесса, принимающего все запросы. Вся логика должна быть написана в рамках единого процесса, который реализует все возможности нашей системы и в итоге становится слишком раздутым. Данное ПО является однопроцессным, и если мы хотим конкурентно обрабатывать множество запросов, то это нужно делать многопоточным образом, создавая для обработки каждого запроса отдельные потоки. Кроме того, более удачным архитектурным решением здесь будет пул потоков с ограниченным количеством элементов.

Ниже перечислены некоторые аспекты конкурентности и синхронизации, о которых необходимо позаботиться.

Если количество запросов существенно возрастает, то для того, чтобы справиться с нагрузкой, следует увеличить количество потоков в пуле. Это буквально требует обновления аппаратного обеспечения и ресурсов компьютера, на котором выполняется главный процесс. Такой подход называется вертикальным масштабированием.

Если в ходе обработки запросов вносятся изменения в разделяемое состояние или хранилище данных, то мы можем легко применить методы синхронизации, зная о том, что потоки имеют доступ к общему пространству памяти. Конечно, это требуется как в случае с разделяемой структурой данных, которую нужно хранить, так и при доступе к удаленному хранилищу данных, которое не является транзакционным.



Локальная многопроцессность

В этом сценарии у нашего программного обеспечения есть несколько процессов, но все они развернуты на одном компьютере. Процессы могут быть однопоточными или содержать внутри себя пул потоков, который позволяет каждому из них обрабатывать сразу несколько запросов.

При увеличении количества запросов мы можем создавать новые процессы вместо новых потоков. Это обычно называется горизонтальным масштабированием. Однако при наличии всего одного компьютера масштабирование должно происходить вертикально, то есть за счет обновления аппаратного обеспечения. Это чревато теми же проблемами, которые упоминались в предыдущем подразделе в контексте вертикального масштабирования многопоточной программы.

Процессы выполняются в конкурентной среде. В целях разделения состояния и синхронизации они могут использовать только многопроцессные методики. Это, безусловно, не настолько удобно, как писать многопоточный код. Кроме того, для разделения состояний процессы могут применять как активные, так и пассивные механизмы.

Многопроцессность на одном компьютере не очень эффективна, и многопоточность, по всей видимости, более удобна с точки зрения усилий, необходимых для написания кода.



Распределенная многопроцессность

В нашем заключительном сценарии мы написали программу, которая состоит из нескольких процессов, выполняемых на разных компьютерах. Все они соединены с помощью сети, и на каждом из них может находиться несколько процессов. Такая конфигурация имеет свои особенности.

При существенном увеличении количества запросов данная система может горизонтально масштабироваться без каких-либо ограничений. Это отличное свойство, позволяющее справляться с пиковыми нагрузками за счет добавления нового потребительского оборудования. Объединение данного оборудования в кластеры, вместо того чтобы использовать мощные серверы, было одной из тех идей, которые позволили компании Google выполнять свои алгоритмы Page Rank и Map Reduce на кластерах компьютеров.

Подходы, мало чем помогают, поскольку у всех у них есть одно важное требование: все процессы должны находиться на одном компьютере. Следовательно, чтобы синхронизировать процессы и предоставить всем им доступ к разделяемым состояниям, нужен совершенно другой набор алгоритмов и методик. В таких распределенных системах следует анализировать и оптимизировать латентность, отказоустойчивость, доступность, согласованность данных и многие другие факторы.

Процессы, размещенные на разных компьютерах, взаимодействуют в пассивной манере с помощью сетевых сокетов, в то время как процессы, принадлежащие одному компьютеру, могут использовать для передачи данных и разделения состояния локальные методы IPC, такие как очереди сообщений, разделяемая память, каналы и т. д.

Адженда

Конкурентность

30 минут

**Синхронизация
потоков**

30 минут

Синхронизация

30 минут

Процессы

30 минут

**Многопоточное
выполнение**

30 минут

**Синхронизация
процессов**

30 минут

Введение

Управляющие механизмы в многопроцессных программах отличаются от тех методик, с которыми мы познакомились при обсуждении многопоточности. Отличается не только память; отдельные факторы существуют лишь в многопроцессных средах, и вы не встретите их в многопоточной программе.

В отличие от потоков, привязанных к одному процессу, сами процессы могут свободно выполняться на любых компьютерах с любыми операционными системами и в любого рода сети, даже в такой большой, как Интернет. Как можно себе представить, это все усложняет. Синхронизировать разные процессы в такой распределенной системе будет нелегко.



Именованные POSIX-семафоры

```
#include <semaphore.h> // Для использования семафоров
#define SHARED_MEM_SIZE 4
// Разделяемый файловый дескриптор, с помощью которого
// мы ссылаемся на объект в разделяемой памяти
int shared_fd = -1;
// Указатель на разделяемый счетчик
int32_t* counter = NULL;
// Указатель на разделяемый семафор
sem_t* semaphore = NULL;
```

Семафоры — основное средство синхронизации разных конкурентных заданий. Пример демонстрирует применение POSIX-семафора в целях устранения гонки данных. В нем используется область разделяемой памяти для хранения переменной с разделяемым счетчиком. Однако на сей раз для синхронизации доступа к счетчику применяются именованные семафоры. Здесь объявляются глобальный счетчик и глобальный указатель на объект семафора, который будет инициализирован позже. Этот указатель позволит родительскому и дочернему процессам иметь синхронизированный доступ к разделяемому счетчику через его указатель.



Именованные POSIX-семафоры

```
void init_control_mechanism() {
    semaphore = sem_open("/sem0", O_CREAT | O_EXCL, 0600, 1);
    if (semaphore == SEM_FAILED) {
        fprintf(stderr, "ERROR: Opening the semaphore failed: %s\n",
                strerror(errno));
        exit(1);
    }
}

void shutdown_control_mechanism() {
    if (sem_close(semaphore) < 0) {
        fprintf(stderr, "ERROR: Closing the semaphore failed: %s\n",
                strerror(errno));
        exit(1);
    }
    if (sem_unlink("/sem0") < 0) {
        fprintf(stderr, "ERROR: Unlinking failed: %s\n",
                strerror(errno));
        exit(1);
    }
}
```

Внутри функций `init_control_mechanism` и `shutdown_control_mechanism` используется API для открытия, закрытия и удаления семафора, подобный интерфейсу разделяемой памяти. Функции `sem_open`, `sem_close` и `sem_unlink` можно считать аналогами `shm_open`, `shm_close` и `shm_unlink`. Но есть одно отличие: функция `sem_open` возвращает указатель на семафор, а не файловый дескриптор.

Именованные POSIX-семафоры

```
void inc_counter() {
    usleep(1);
    sem_wait(semaphore); // Return value should be checked.
    int32_t temp = *counter;
    usleep(1);
    temp++;
    usleep(1);
    *counter = temp;
    sem_post(semaphore); // Return value should be checked.
    usleep(1);
}
```

Критический участок, в котором инкрементируется разделяемый счетчик в Unix-подобных операционных системах существует два вида именованных семафоров: семафоры System V и POSIX-семафоры. У последних благодаря своей производительности и удобному API их репутация куда лучше. Ниже приводится вопрос на сайте [Stack Overflow](#), в котором хорошо описаны различия между семафорами System V и POSIX-семафорами.

Если говорить о работе с семафорами, то Microsoft Windows не является POSIX-совместимой ОС. У нее есть собственный API для создания и управления семафорами.



Именованные POSIX-семафоры

```
int main(int argc, char **argv) {  
    // Родительский процесс должен инициализировать разделяемый ресурс  
    init_shared_resource();  
    // Родительский процесс должен инициализировать механизм управления  
    init_control_mechanism();  
    ...  
    // Only parent process needs to shut down the shared resource  
    // and the employed control mechanism  
    if (pid) {  
        shutdown_shared_resource();  
        shutdown_control_mechanism();  
    }  
}
```

```
$ gcc main.c -lrt -lpthread -o main.out  
$ ./main.out  
Shared memory fd is: 3  
The memory region is truncated.  
Parent process sees the counter as 1.  
Child process sees the counter as 2.  
Child process finished with status 0.
```

В показанном выше выводе все понятно. Иногда первым получает ресурсы процессора и инкрементирует счетчик дочерний процесс, а иногда — родительский. Они не могут зайти на критический участок одновременно, поэтому наш код соблюдает целостность данных в отношении разделяемого счетчика.

Именованные мьютексы

В многопоточных программах POSIX-мьютексы работают довольно просто. Однако в многопроцессных средах все немного иначе. Чтобы мьютекс можно было использовать в разных процессах, его необходимо определить там, где он будет им доступен.

Лучшее место подобного рода — область разделяемой памяти. Следовательно, чтобы получить мьютекс, который работает в многопроцессной среде, его нужно распределить именно в этой области.



Первый пример

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/wait.h>
```

```
#include <pthread.h> // Для использования функций pthread_mutex_*
#define SHARED_MEM_SIZE 4
int shared_fd = -1; // Разделяемый файловый дескриптор для обращения к
объекту в разделяемой памяти
int mutex_shm_fd = -1; // Разделяемый файловый дескриптор для обращения
к объекту в разделяемой памяти мьютекса
int32_t *counter = NULL; // Указатель на разделяемый счетчик
pthread_mutex_t *mutex = NULL; // Указатель на разделяемый мьютекс
```

Здесь мы объявили:

- глобальный файловый дескриптор для обращения к области разделяемой памяти, в которой должна храниться переменная разделяемого счетчика;
- глобальный файловый дескриптор для области разделяемой памяти, в которой хранится разделяемый мьютекс;
- указатель на разделяемый счетчик;
- указатель на разделяемый мьютекс.

Для борьбы с потенциальным состоянием гонки в нем применяются именованные мьютексы. Поскольку каждый объект разделяемой памяти имеет глобальное имя, мьютекс, размещаемый в этой области, можно считать именованным, и обращаться к нему могут другие процессы в системе.

Первый пример

```
void init_control_mechanism() {  
    // Открываем разделяемую память мьютекса  
    mutex_shm_fd = shm_open("/mutex0", O_CREAT | O_RDWR, 0600);  
    if (mutex_shm_fd < 0)  
        // Выделяем и усекаем разделяемую область памяти мьютекса  
        if (ftruncate(mutex_shm_fd, sizeof(pthread_mutex_t)) < 0)  
            // Отображаем разделяемую память мьютекса  
            void *map = mmap(0, sizeof(pthread_mutex_t),  
                             PROT_READ | PROT_WRITE, MAP_SHARED, mutex_shm_fd, 0);  
    if (map == MAP_FAILED)  
        mutex = (pthread_mutex_t *)map;  
    // Инициализируем объект мьютекса  
    int ret = -1;  
    pthread_mutexattr_t attr;  
    if ((ret = pthread_mutexattr_init(&attr)))  
    if ((ret = pthread_mutexattr_setpshared(&attr, PTHREAD_PROCESS_SHARED)))  
    if ((ret = pthread_mutex_init(mutex, &attr)))  
    if ((ret = pthread_mutexattr_destroy(&attr)))  
}
```

Мьютекс, выделенный в разделяемой памяти нужно инициализировать. Для этого вызываем функцию `pthread_mutex_init` и передаем ей атрибуты, говорящие что объект мьютекса должен быть разделяемым и доступным для других процессов. Иначе мьютекс не будет работать в многопроцессной среде, даже если поместить его в область разделяемой памяти. В функции `init_control_mechanism`, мы установили атрибут `PTHREAD_PROCESS_SHARED`, чтобы сделать мьютекс разделяемым.

Первый пример

```
void shutdown_control_mechanism() {
    int ret = -1;
    if ((ret = pthread_mutex_destroy(mutex))) {
        fprintf(stderr, "ERROR: Failed to destroy mutex: %s\n", strerror(ret));
        exit(1);
    }
    if (munmap(mutex, sizeof(pthread_mutex_t)) < 0) {
        fprintf(stderr, "ERROR: Unmapping the mutex failed: %s\n", strerror(errno));
        exit(1);
    }
    if (close(mutex_shm_fd) < 0) {
        fprintf(stderr, "ERROR: Closing the mutex failed: %s\n", strerror(errno));
        exit(1);
    }
    if (shm_unlink("/mutex0") < 0) {
        fprintf(stderr, "ERROR: Unlinking the mutex failed: %s\n", strerror(errno));
        exit(1);
    }
}
```

В функции `shutdown_control_mechanism` мы уничтожаем объект мьютекса и затем закрываем и удаляем соответствующую область разделяемой памяти. Это ничем не отличается от уничтожения обычного объекта разделяемой памяти.

Первый пример

```
void init_shared_resource() { /*как в предыдущем примере*/ }
void shutdown_shared_resource() { /*как в предыдущем примере*/}
void inc_counter() {
    usleep(1);
    pthread_mutex_lock(mutex); // Нужно проверить возвращаемое значение
    int32_t temp = *counter;
    usleep(1);
    temp++;
    usleep(1);
    *counter = temp;
    pthread_mutex_unlock(mutex); // Нужно проверить возвращаемое значение
    usleep(1);
}
int main(int argc, char **argv) { /*как в предыдущем примере*/ }
```

Мы заменили лишь управляющий механизм, не трогая остальную логику.

В функции `inc_counter` объект мьютекса используется точно так же, как это делалось в многопоточной программе. API абсолютно аналогичен; он спроектирован таким образом, чтобы его можно было применять как в многопоточных, так и в многопроцессных средах. Эта замечательная особенность POSIX-мьютексов позволяет нам использовать один и тот же код, независимо от того, что работает с этими объектами: потоки или процессы. Хотя, конечно, инициализация и уничтожение могут отличаться.

Второй пример

Именованные объекты разделяемой памяти и мьютексы можно применять в любом процессе, запущенном в системе. И этот процесс не обязательно должен быть клонированным. В примере мы рассмотрим совместное использование разделяемого мьютекса и разделяемой памяти для одновременного завершения нескольких процессов, которые работают параллельно. Мы хотим, чтобы при нажатии Ctrl+C в одном из процессов все они немедленно прекращали работу. Пример 30_24.



Этап 1: глобальные определения

(строка 1-31) Мы будем работать с одним исходным файлом, который можно будет скомпилировать и многократно запустить для создания нескольких процессов. В целях синхронизации своего выполнения эти процессы используют области разделяемой памяти. Один из них выбирает владелец этих областей; он будет отвечать за их создание и уничтожение. Другие процессы просто применяют уже созданную разделяемую память. На первом этапе мы объявим глобальные объекты, которые понадобятся нам на разных участках кода. Позже они будут инициализированы.

Обратите внимание: глобальные переменные, объявленные в примере (такие как `mutex`), не разделяются между процессами. Они существуют в адресном пространстве каждого отдельного процесса, но привязываются к разным областям разделяемой памяти.

Разделяемый флаг позволит уведомлять процессы о сигнале отмены. Отмечу, что в данном примере для ожидания момента, когда флаг отмены станет равен `true`, будет использоваться холостой цикл.

Здесь у нас есть два отдельных объекта разделяемой памяти: один для флага отмены, а другой — для мьютекса, который защищает данный флаг.

Я мог бы создать единую структуру и определить в ней оба объекта в виде полей, что позволило бы хранить их в одной области разделяемой памяти. Но я решил использовать для этого разные области.

Одним из важных нюансов при работе с объектами разделяемой памяти в этом примере является то, что очистку должен производить процесс, который их изначально создал и инициализировал. Поскольку все процессы используют один и тот же код, нам нужно каким-то образом определить владельца объекта. Таким образом, когда мы дойдем до очистки, удалить объект сможет только его владелец. В связи с этим нам пришлось объявить две булевы переменные: `mutex_owner` и `cancel_flag_shm_owner`.

Этап 2: разделяемая память флага отмены

(строка 32-121)

init_shared_resource(): процедура инициализации области разделяемой памяти, отведенной флагу отмены. Каждый новый процесс должен проверять, был ли объект разделяемой памяти создан другим процессом.

Обратите внимание: в этом примере создание новых процессов обходится без функции `fork`, и пользователь может самостоятельно создавать новые процессы в своей командной оболочке.

В связи с этим новый процесс сначала пытается открыть область разделяемой памяти, предоставляя один лишь флаг `O_RDWR`. Успех будет признаком того, что текущий процесс не владеет данной областью, поэтому на следующем шаге привязывает его. Если создание пройдет успешно, то текущий процесс становится владельцем области разделяемой памяти и переходит к ее усечению и привязке.

В описанном выше сценарии существует небольшой шанс, что другой процесс создаст ту же область разделяемой памяти прямо между двумя вызовами `shm_open`, в результате чего второй из них завершится неудачей. Флаг `O_EXCL` не дает текущему процессу создать объект, если тот уже существует.

shutdown_shared_resource(): Освобождение ресурсов, выделенных для разделяемой памяти флага отмены. Логика освобождения объекта разделяемой памяти, представленная в данном листинге, очень похожа на ту, которую мы видели в предыдущих примерах. Но здесь есть одно отличие: удалить объект разделяемой памяти может только владеющий им процесс.

Обратите внимание: прежде чем удалять объект, владелец ждет 1 секунду, чтобы другие процессы успели освободить свои ресурсы. Эта задержка обычно не требуется, поскольку в большинстве POSIX-совместимых систем объект разделяемой памяти остается на месте, пока не завершатся все процессы, которые зависят от него.

Этап 4: установка флага отмены

(строка 253-265)

Синхронизированные функции, которые считывают и устанавливают флаг отмены, защищенный разделяемым мьютексом

```
bool_t is_canceled() {  
    pthread_mutex_lock(mutex); // Нужно проверить возвращаемое значение  
    bool_t temp = *cancel_flag;  
    pthread_mutex_unlock(mutex); // Нужно проверить возвращаемое значение  
    return temp;  
}  
void cancel() {  
    pthread_mutex_lock(mutex); // Нужно проверить возвращаемое значение  
    *cancel_flag = TRUE;  
    pthread_mutex_unlock(mutex); // Нужно проверить возвращаемое значение  
}
```

Эти две функции обеспечивают синхронизированный доступ к разделяемому флагу отмены. Функция `is_canceled` используется для проверки значения флага, а функция `cancel` — для его установки. Как видите, обе они защищены одним и тем же разделяемым мьютексом.

Этап 5: главная функция

(строка 266-287)

Функция `main` и обработчик сигналов

```
void sigint_handler(int signo) {
    fprintf(stdout, "\nHandling INT signal: %d ...\n", signo);
    cancel();
}

int main(int argc, char **argv) {
    signal(SIGINT, sigint_handler);
    // Родительский процесс должен инициализировать разделяемый ресурс
    init_shared_resource();
    // Родительский процесс должен инициализировать механизм управления
    init_control_mechanism();
    while (!is_canceled()) {
        fprintf(stdout, "Working ...\n");
        sleep(1);
    }
    fprintf(stdout, "Cancel signal is received.\n");
    shutdown_shared_resource();
    shutdown_control_mechanism();
    return 0;
}
```

Логика внутри функции `main` выглядит вполне понятно. Она инициализирует разделяемые флаг и мьютекс, после чего входит в холостой цикл и ждет, пока флаг отмены будет равен `true`. В конце она освобождает все разделяемые ресурсы и завершает работу.

Этап 5: главная функция

Новое: использование функции `signal`, которая назначает обработчик для определенного набора сигналов.

Сигналы — один из механизмов, доступный во всех POSIX-совместимых операционных системах, с помощью которого процессы могут слать друг другу информацию в рамках одной ОС. **Терминал** — обычный процесс, с которым взаимодействуют пользователи и который позволяет отправлять сигналы другим процессам. Нажать `Ctrl+C` — удобный способ послать сигнал `SIGINT` активному процессу, запущенному в терминале.

Сигнал `SIGINT` представляет собой сигнал прерывания, который может получить процесс. Ранее мы назначили функцию `sigint_handler` в качестве обработчика сигнала `SIGINT`. Иными словами, эта функция вызывается каждый раз, когда процесс получает этот сигнал. Если `SIGINT` не обработать, то будет выполнена процедура по умолчанию, которая завершит процесс, но ее можно переопределить с помощью обработчиков сигналов, как показано выше.

Существует множество способов, как отправить процессу сигнал `SIGINT`, но один из самых простых — нажать сочетание клавиш `Ctrl+C` на клавиатуре, в результате которого сигнал будет сразу же получен. Внутри обработчика сигнала мы присваиваем разделяемому флагу отмены значение `true`, после чего все процессы начинают выходить из своих холостых циклов.

```
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/D
```

```
mikhail@DESKTOP-R6I9BAG: /mnt/c/Users/Mfili/D
```

```
mikhail@DESKTOP-R6I9BAG: /mnt/c/Users/Mfili/D
```

Именованные условные переменные

Чтобы использовать условные переменные POSIX в многопроцессной системе, их, как и именованные POSIX-мьютексы, необходимо выделять в области разделяемой памяти. В примере далее показано, как это делается; мы создадим ряд процессов, которые будут выполнять отсчет в определенном порядке. Каждая условная переменная должна использоваться в связке с объектом мьютекса, защищающим ее. Поэтому будет три области разделяемой памяти: для разделяемого счетчика, для разделяемой именованной условной переменной и для разделяемого именованного мьютекса, который будет ее защищать.

Обратите внимание: вместо трех разных областей разделяемой памяти мы могли бы использовать одну общую. Для этого можно было бы определить структуру, которая включает в себе все необходимые объекты. Пример состоит из ряда процессов, которые должны выполнять отсчет в порядке возрастания. Каждому процессу назначается номер, начиная с 1 и заканчивая количеством процессов. Данный номер обозначает приоритет. Первоочередное право отсчета отдается процессу с наименьшим номером; только когда он закончит, следующий процесс сможет выполнить свой отсчет и завершить работу. Это значит, первым отсчет выполняет процесс с номером 1, даже если был создан последним.

Поскольку у нас будет три разные области разделяемой памяти, каждую из которых нужно отдельно инициализировать и освободить, нам пришлось бы дублировать много кода, если бы мы использовали тот же подход, что и в предыдущих примерах. Сделаем код более лаконичным и организованным, а также оформим повторяющиеся участки в виде функций с помощью объектно-ориентированного стиля.

Мы определим общий родительский класс для всех областей разделяемой памяти, а также по одному дочернему классу для разделяемого счетчика, разделяемого именованного мьютекса и

разделяемой именованной условной переменной. У каждого из этих классов будет своя пара заголовочных и исходных файлов, и все они будут использоваться в главной функции нашего прим

Этап 1: класс разделяемой памяти

Публичный интерфейс класса разделяемой памяти `shared_mem.h`

В этом листинге содержатся объявления (публичный API), необходимые для работы с объектом разделяемой памяти. Функции `shared_mem_getptr`, `shared_mem_isowner` и `shared_mem_setowner` представляют собой поведение данного класса.

Определения всех функций, принадлежащих классу разделяемой памяти (`shared_mem.c`)

Как видите, мы просто скопировали код, написанный для работы с разделяемой памятью в предыдущих примерах. Структура `shared_mem_t` инкапсулирует все, что нам нужно для обращения к объекту разделяемой памяти POSIX. Обратите внимание на глобальную булеву переменную `process_owner`. Она определяет, является ли текущий процесс владельцем всех областей разделяемой памяти. Она устанавливается только однажды.

Этап 2: класс разделяемого 32-битного целочисленного счетчика

Публичный интерфейс класса разделяемого счетчика (`shared_int32.h`)

В этом файле содержится определение класса разделяемого счетчика, который представляет собой 32-битное целое число. Этот класс наследуется от класса разделяемой памяти. Как вы могли заметить, для реализации наследования мы используем только второй подход (28 лекция).

Определения всех функций в классе разделяемого счетчика (`shared_int32.c`)

В примере представлены реализации функций, объявленных выше.

Как видите, благодаря наследованию наш код получился намного более компактным. Все операции по управлению соответствующим объектом разделяемой памяти доступны в поле `shm` структуры `shared_int32_t`.

Этап 3: класс разделяемого мьютекса

Публичный интерфейс класса разделяемого мьютекса (shared_mutex.h)

Представленный выше класс, как и ожидалось, содержит три операции: `shared_mutex_lock`, `shared_mutex_unlock` и `shared_mutex_make_consistent`. Но есть один нюанс: операция `shared_mutex_make_consistent` доступна только в системах POSIX, которые не принадлежат к семейству macOS (Apple). Дело в том, что системы Apple не поддерживают устойчивые мьютексы. Мы еще вернемся к этому в следующих абзацах. Обратите внимание: здесь используется макрос `__APPLE__`, позволяющий определить, компилируется ли код в системе Apple.

Определения всех функций, которые находятся в классе разделяемого именованного мьютекса (shared_mutex.c)

В данном коде выполняются инициализация и освобождение POSIX-мьютекса, а также предоставляется доступ к таким тривиальным операциям, как блокировка и разблокировка. Все остальное, касающееся объекта разделяемой памяти, находится в родительском классе. Это одно из преимуществ использования наследования.

Обратите внимание: в функции-конструкторе `shared_mutex_ctor` мы делаем мьютекс разделяемым (`PTHREAD_PROCESS_SHARED`), чтобы он был доступен для всех процессов. В многопроцессном ПО это совершенно необходимо. Кроме того, в системах, не принадлежащих к семейству Apple, мы идем еще дальше и помечаем мьютекс как устойчивый (`PTHREAD_MUTEX_ROBUST`). Когда процесс завершается неожиданно, обычный мьютекс, заблокированный им, переходит в несогласованное состояние. Однако устойчивый мьютекс в этой ситуации можно опять сделать согласованным. Следующий процесс, который обычно хочет разблокировать мьютекс, может заблокировать его только после его согласования. В функции `shared_mutex_lock` показано, как это делается. В системах Apple данной возможности нет.

Этап 5: основная логика

Главная функция в примере (*main.c*)

Как видите, программа принимает аргумент, обозначающий ее номер. Узнав его, процесс начинает инициализацию разделяемого счетчика, разделяемого мьютекса и разделяемой условной переменной. Затем заходит на критический участок, защищенный разделяемым мьютексом.

Внутри цикла процесс ждет, когда счетчик станет равным его номеру. Поскольку ожидание длится 5 секунд, время теоретически может истечь, в результате чего функция `shared_cond_timedwait` завершится. Это фактически означает, что условная переменная не была уведомлена на протяжении этих 5 секунд. Затем процесс проверяет условие и использует еще один пятисекундный период ожидания. Так продолжается до тех пор, пока не придет его очередь. Когда это случится, процесс выведет свой номер, инкрементирует разделяемый счетчик и уведомит о данном изменении остальные ожидающие процессы, разослав им сигналы об объекте условной переменной. И только после этого он приступит к завершению работы.

Тем временем, если пользователь нажмет Ctrl+C, обработчик сигнала, определенный в рамках основной логики, установит локальный флаг `int_received` и, как только процесс, находящийся внутри главного цикла, покинет функцию `shared_mutex_timedwait`, заметит сигнал прерывания, и цикл будет завершен.

```
$ gcc -c shared_mem.c -o shared_mem.o
$ gcc -c shared_int32.c -o shared_int32.o
$ gcc -c shared_mutex.c -o shared_mutex.o
$ gcc -c shared_cond.c -o shared_cond.o
$ gcc -c main.c -o main.o
$ gcc shared_mem.o shared_int32.o shared_mutex.o shared_cond.o main.o -lpthread -lrt -o main.out
```

Этап 5: основная логика

ПРОБЛЕМЫ10

ВЫХОДНЫЕ ДАННЫЕ

КОНСОЛЬ ОТЛАДКИ

ПОРТЫ

ТЕРМИНАЛ

The shared memory /mutex0 is deleted.
The shared memory /counter0 is unmapped.
The shared memory /counter0 is closed.
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lectio
ns_PowerPoint/lection_31_25\$./a.out 3
My number is 3!
WARN: The shared memory /counter0 does not exist.
The shared memory /counter0 is created and opened.
WARN: The shared memory /mutex0 does not exist.
The shared memory /mutex0 is created and opened.
WARN: The shared memory /cond0 does not exist.
The shared memory /cond0 is created and opened.
Waiting for the signal, just for 5 seconds ...
Checking condition ...
Waiting for the signal, just for 5 seconds ...

/Users/Mfili/Desktop/learningc++/imperative_programming/lectio
ns_PowerPoint/lection_31_25\$
out 2.
My number is 2!
The shared memory /counter0 is opened.
The shared memory /mutex0 is opened.
The shared memory /cond0 is opened.
Waiting for the signal, just for 5 seconds ...
Checking condition ...
Waiting for the signal, just for 5 seconds ...
Checking condition ...
Waiting for the signal, just for 5 seconds ...
Checking condition ...
Waiting for the signal, just for 5 seconds ...
Checking condition ...

PS C:\Users\Mfili\Desktop\learningc++\imperative_programming\lections_PowerPoint\lection_31_25> wsl
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lectio
ns_PowerPoint/lection_31_25\$ 1
1: command not found
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lectio
ns_PowerPoint/lection_31_25\$./a.out 1
My number is 1!
The shared memory /counter0 is opened.
The shared memory /mutex0 is opened.
The shared memory /cond0 is opened.
My turn! 1 ...
The shared memory /cond0 is unmapped.
The shared memory /cond0 is closed.
The shared memory /mutex0 is unmapped.
The shared memory /mutex0 is closed.

Строка 7, столбец 26 Пробелов: 4 UTF-8 CRLF {} C windows-qcc-x64 Go Live

ПРОБЛЕМЫ10

ВЫХОДНЫЕ ДАННЫЕ

КОНСОЛЬ ОТЛАДКИ

ПОРТЫ

ТЕРМИНАЛ

Waiting for the signal, just for 5 seconds ...
Checking condition ...
Waiting for the signal, just for 5 seconds ...
Checking condition ...
My turn! 3 ...
The shared memory /cond0 is unmapped.
The shared memory /cond0 is closed.
The shared memory /cond0 is deleted.
The shared memory /mutex0 is unmapped.
The shared memory /mutex0 is closed.
The shared memory /mutex0 is deleted.
The shared memory /counter0 is unmapped.
The shared memory /counter0 is closed.
The shared memory /counter0 is deleted.
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lectio
ns_PowerPoint/lection_31_25\$

Checking condition ...
Waiting for the signal, just for 5 seconds ...
Checking condition ...
Waiting for the signal, just for 5 seconds ...
Checking condition ...
Waiting for the signal, just for 5 seconds ...
Checking condition ...
My turn! 2 ...
The shared memory /cond0 is unmapped.
The shared memory /cond0 is closed.
The shared memory /mutex0 is unmapped.
The shared memory /mutex0 is closed.
The shared memory /counter0 is unmapped.
The shared memory /counter0 is closed.
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lectio
ns_PowerPoint/lection_31_25\$

1: command not found
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lectio
ns_PowerPoint/lection_31_25\$./a.out 1
My number is 1!
The shared memory /counter0 is opened.
The shared memory /mutex0 is opened.
The shared memory /cond0 is opened.
My turn! 1 ...
The shared memory /cond0 is unmapped.
The shared memory /cond0 is closed.
The shared memory /mutex0 is unmapped.
The shared memory /mutex0 is closed.
The shared memory /counter0 is unmapped.
The shared memory /counter0 is closed.
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/Desktop/learningc++/imperative_programming/lectio
ns_PowerPoint/lection_31_25\$

Строка 49, столбец 6 Пробелов: 4 UTF-8 CRLF {} C windows-q

132

Распределенное управление конкурентностью

До сих пор речь всегда шла о локальном ПО. Однако настоящие программные продукты обычно этим не ограничиваются. Помимо локальных систем существуют еще и распределенные. В них процессы разбросаны по разным устройствам и взаимодействуют по сети. Некоторые аспекты систем с распределенными процессами создают трудности, отсутствующие (или не проявляющиеся в такой степени) в централизованных или локальных системах. Кратко их обсудим.

- В распределенной программной системе вы, скорее всего, имеете дело с параллелизмом, а не с конкурентностью.
- Не все процессы внутри распределенной программной системы написаны на одном и том же языке.
- В распределенной системе в качестве канала между процессами, находящимися на разных компьютерах, всегда используется сеть.
- Наличие сети между процессами вызывает латентность. Латентность попросту означает, что по многим причинам, связанным с сетевой инфраструктурой, процесс может не получить сообщение сразу.
- Наличие сети между процессами также приводит к проблемам с безопасностью.
- Помимо проблем с латентностью и безопасностью, у вас могут возникнуть трудности с доставкой, которые в локальных системах случаются намного реже. Чтобы обработать сообщения, их нужно сначала доставить.

Распределенное управление конкурентностью

Перечисленные выше отличия и потенциальные проблемы оказались достаточным поводом для изобретения новых способов синхронизации процессов и различных компонентов в огромных распределенных системах. В целом распределенную систему можно сделать транзакционной и синхронизированной двумя путями.

- Через централизованную синхронизацию процессов. Этот подход подразумевает наличие центрального процесса (или узла), который управляет остальными компонентами. Все другие процессы в системе должны постоянно с ним взаимодействовать и запрашивать его разрешения для захода на свои критические участки.
- Через распределенную (пиринговую) синхронизацию процессов. Построение инфраструктуры синхронизации процессов, не имеющей центрального узла, — непростая задача. На самом деле в этой области ведутся активные исследования. Кроме того, существуют некоторые узкоспециализированные алгоритмы.

Бонус

Параллельные
алгоритмы

20 минут

Введение

Мы изучали только последовательные (serial), пригодные для выполнения на однопроцессорном компьютере, который в каждый момент времени выполняет только одну инструкцию. Сейчас мы расширим нашу алгоритмическую модель параллельными алгоритмами (parallel algorithms), которые могут выполняться на многопроцессорных компьютерах, допускающих одновременное выполнение нескольких команд.

Относительно недорогие настольные и переносные компьютеры оснащены одним многоядерным процессором, в который входят несколько "ядер", которые сами по себе являются полноценными процессорами с доступом к общей памяти. В той или иной форме многопроцессорные компьютеры существуют уже десятилетия. Но несмотря на то, что модель машины с произвольным доступом для последовательных вычислений появилась и была принята еще на ранней стадии развития компьютерных наук, до сих пор ни одна модель для параллельных вычислений не получила широкого признания. Основная причина этого в том, что производители не договорились о единой архитектурной модели для параллельных компьютеров. Например, одни параллельные компьютеры оснащены совместно используемой памятью (shared memory), где каждый процессор может непосредственно обращаться к любой ячейке памяти. Другие параллельные компьютеры используют распределенную память (distributed memory), где каждый процессор имеет собственную память, и для доступа одного процессора к памяти другого между процессорами должны передаваться явные сообщения.



Динамическое многопоточное программирование

Важным классом параллельных платформ является динамическая многопоточность (dynamic multithreading). Динамическая многопоточность позволяет программисту указывать уровень параллелизма в приложении, не беспокоясь о коммуникационных протоколах, сбалансированности загрузки и других неприятностях программирования со статическими потоками. Хотя функциональность сред с динамической многопоточностью продолжает развиваться, почти все они поддерживают две возможности: вложенный параллелизм и параллельные циклы. Ключевым моментом этой модели является то, что программист должен указывать только логическую параллельность вычислений, потоки параллельной платформы и распределение вычислений между ними.

Наша модель динамической многопоточности имеет несколько важных преимуществ. Она является простым расширением нашей последовательной модели. Мы можем описать многопоточный алгоритм, добавив в псевдокод только три ключевых слова: `parallel`, `spawn` и `sync`. Кроме того, если мы удалим эти слова из многопоточного псевдокода, то получим текст последовательного псевдокода для решения той же задачи, который мы будем называть сериализацией (serialization) многопоточного алгоритма. Она обеспечивает теоретически ясный способ количественного описания параллелизма на основе понятий работы (work) и интервала (span).



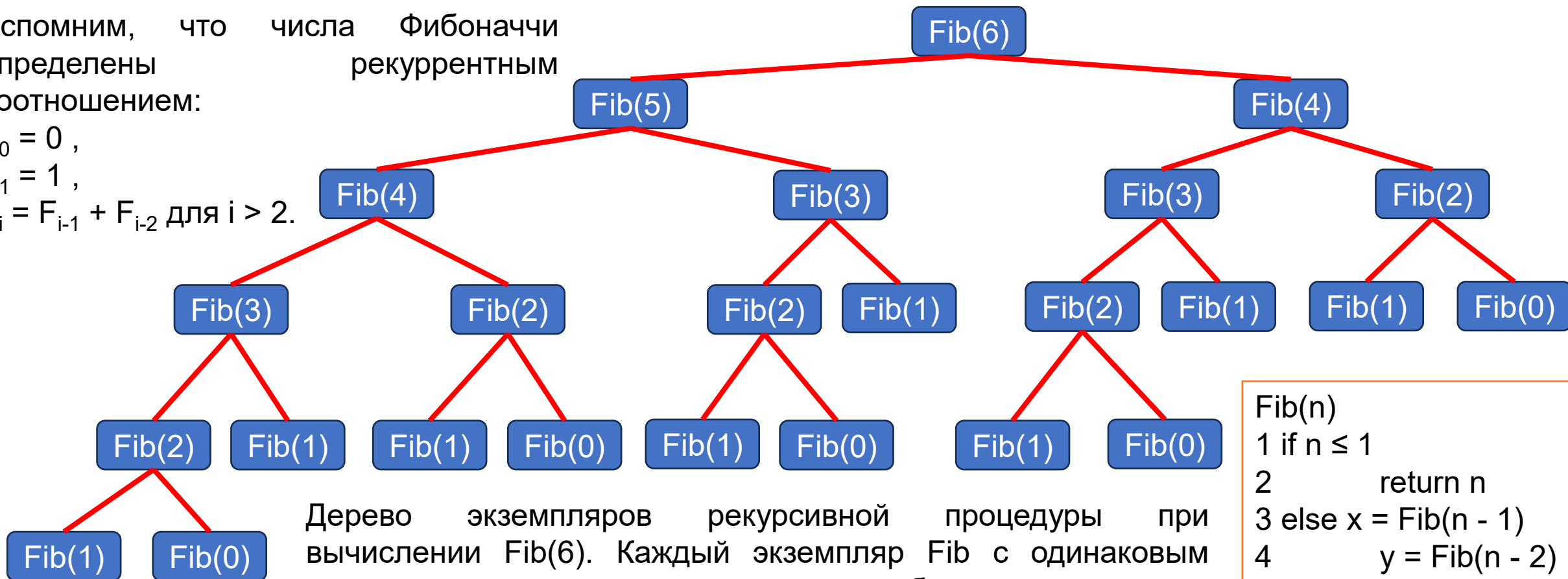
Основы динамической многопоточности

Вспомним, что числа Фибоначчи определены рекуррентным соотношением:

$$F_0 = 0,$$

$$F_1 = 1,$$

$$F_i = F_{i-1} + F_{i-2} \text{ для } i > 2.$$



```

Fib(n)
1 if n ≤ 1
2     return n
3 else x = Fib(n - 1)
4     y = Fib(n - 2)
5     return x + y
    
```

Основы динамической многопоточности

Поскольку процедура Fib не использует технологию запоминания (memoization), второй вызов Fib(4) вновь выполняет всю работу, уже выполненную первым вызовом.

Обозначим через $T(n)$ время работы процедуры Fib(n). Поскольку FIB(n) содержит два рекурсивных вызова плюс константное количество дополнительной работы, мы получим рекуррентное соотношение

$$T(n) = T(n - 1) + T(n - 2) + O(1).$$

Это рекуррентное соотношение имеет решение $T(n) = \Theta(F_n)$, что можно показать с помощью метода подстановки. В качестве гипотезы индукции примем, что $T(n) \leq aF_n - b$, где $a > 1$ и $b > 0$ представляют собой константы. При

подстановке получим

$$T(n) \leq (aF_{n-1} - b) + (aF_{n-2} - b) + O(1) = a(F_{n-1} + F_{n-2}) - 2b + O(1) = aF_n - b - (b - \Theta(1)) \leq aF_n - b,$$

если выберем b достаточно большим для доминирования над константой в $\Theta(1)$. Затем мы можем выбрать a достаточно большим для удовлетворения начальному условию. Аналитическая граница

$$T(n) = \Theta(\phi^n), \quad (30.1)$$

где $\phi = \frac{1 + \sqrt{5}}{2}$ представляет собой золотое сечение. Поскольку F_n растет с ростом n экспоненциально, эта процедура представляет собой очень медленный способ вычисления чисел Фибоначчи.

Хотя процедура Fib - не лучший, если не худший способ вычисления чисел Фибоначчи, она представляет собой хороший пример для иллюстрации ключевых концепций анализа многопоточных алгоритмов. Заметим, что в Fib(n) два рекурсивных вызова, Fib(n - 1) и Fib(n - 2), в строках 3 и 4 являются независимыми один от другого: они могут быть вызваны в любом порядке, и вычисления одной процедуры никак не влияют на вычисления другой. Таким образом, эти два рекурсивных вызова могут работать параллельно.

Основы динамической многопоточности

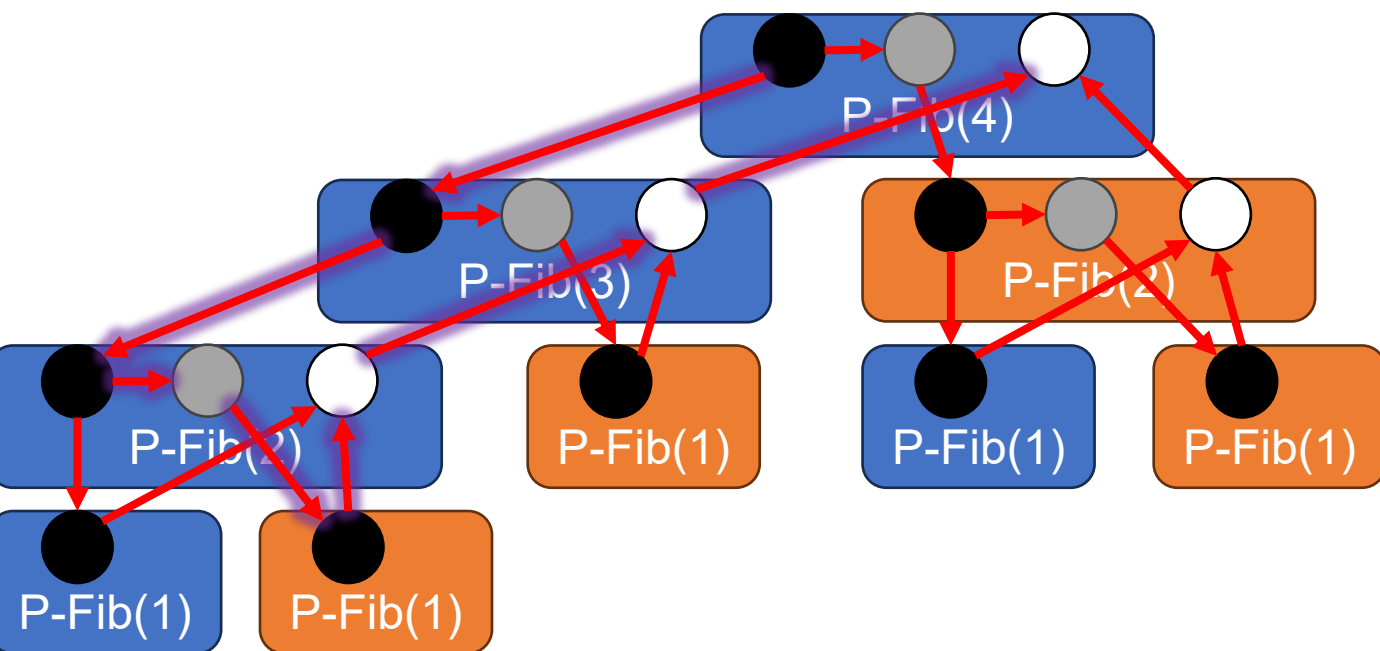
Расширим используемый в книге псевдокод, добавив новые ключевые слова, а именно - **spawn** и **sync**. Вот как можно переписать процедуру FIB с использованием динамической многопоточности.

Обратите внимание, что если мы уберем ключевые слова **spawn** и **sync** из псевдокода P-Fib, то полученный в результате псевдокод будет идентичен FIB. Определим сериализацию (serialization) многопоточного алгоритма как последовательный алгоритм, получаемый при удалении многопоточных ключевых слов **spawn** и **sync** (а когда мы изучим параллельные циклы, еще и **parallel**).

```
P-Fib(n)
1 if n ≤ 1
2     return n
3 else x = spawn P-Fib(n - 1)
4     y = P-Fib(n - 2)
5     sync
6     return x + y
```

Вложенный параллелизм осуществляется, когда ключевое слово **spawn** предшествует вызову процедуры, как в строке 3. Семантика запуска (spawn) отличается от обычного вызова процедуры тем, что экземпляр процедуры, выполняющий запуск (родительская процедура), может продолжать работать параллельно с запущенной дочерней подпрограммой, вместо того чтобы ожидать завершения ее работы, как это обычно делается при последовательном выполнении. Однако ключевое слово **spawn** не требует от процедуры обязательного параллельного с запущенной дочерней подпрограммой выполнения; оно лишь разрешает таковое. Ключевые слова для параллельных вычислений выражают логический параллелизм (logical parallelism) вычислений, указывая, какие части вычислений могут выполняться параллельно. Во время выполнения программы планировщик (scheduler) определяет, какие подвычисления в действительности могут выполняться параллельно, назначая их доступным процессорам.

Модель многопоточного выполнения



Ориентированный ациклический граф, представляющий вычисление P-Fib(4). Каждый круг представляет один фрагмент вычисления, причем черные круги представляют либо базовый случай, либо часть процедуры (экземпляр) до запуска P-Fib($n - 1$) в строке 3, серые круги представляют часть процедуры, которая вызывает P-Fib($n - 2$) в строке 4 перед ключевым словом `sync` в строке 5, где процедура приостанавливается в ожидании возврата запущенной процедуры P-Fib($n - 1$), а белые круги представляют часть процедуры после ключевого слова `sync`, в которой выполняется сложение значения x и y перед точкой, в которой происходит возврат полученного результата.

Каждая группа фрагментов, принадлежащих одной и той же процедуре, ограничена скругленным прямоугольником, синим для запускаемых процедур и оранжевым – для вызываемых. Ребра запуска и вызова направлены вниз, продолжения выполнения – по горизонтали вправо, а ребра возврата – вверх. Полагая, что каждый фрагмент требует единицы времени, вся работа составляет 17 единиц, поскольку имеется 17 фрагментов, а интервал равен 8 единицам, поскольку критический путь (указанный выделенными штриховкой ребрами) содержит 8 фрагментов.

Модель многопоточного выполнения

Мы будем изучать выполнение многопоточных алгоритмов на *идеальном параллельном компьютере* (ideal parallel computer), состоящем из множества процессоров и *последовательно согласованной* (sequentially consistent) совместно используемой памяти. Последовательная согласованность означает, что совместно используемая память, в которую в действительности процессорами одновременно может выполняться много как загрузок, так и чтений, дает одни и те же результаты как и в случае, если на каждом шаге будет выполняться только одна команда одного процессора. Иначе говоря, память ведет себя так, как если бы команды выполнялись последовательно в некотором глобальном линейном порядке, сохраняющем индивидуальный порядок выполнения команд каждым процессором. В случае динамических многопоточных вычислений, автоматически распределяемых по процессорам параллельной платформой, совместно используемая память ведет себя так, как если бы команды многопоточных вычислений чередовались таким образом, что образовывали бы линейный порядок, сохраняющий частичный порядок ориентированного ациклического графа вычислений. В зависимости от планировщика упорядочение может отличаться при разных запусках одной и той же программы, но поведение любого выполнения программы может быть понято в предположении, что команды выполняются в некотором линейном порядке, согласованном с ориентированным ациклическим графом вычислений.

В дополнение к сделанным семантическим предположениям модель идеального параллельного компьютера делает также некоторые предположения, связанные с производительностью. В частности, предполагается, что каждый процессор машины обладает одной и той же вычислительной мощностью, а стоимость планирования игнорируется. Оказывается, что для алгоритмов с достаточной степенью "параллелизма" на практике накладные расходы планирования обычно минимальны.

Меры производительности

Измерять теоретическую эффективность многопоточного алгоритма можно с использованием двух метрик: "работы" (work) и "интервала" (span). **Работа** многопоточного вычисления представляет собой общее время выполнения всего вычисления на одном процессоре. Другими словами, работа представляет собой сумму времен, взятую по всем фрагментам. В ориентированном ациклическом графе вычислений, в котором каждый фрагмент требует единичного времени, работа равна просто количеству вершин ориентированного ациклического графа. Интервал представляет собой наибольшее время выполнения фрагментов вдоль произвольного пути в ориентированном ациклическом графе. Опять же, в случае ориентированного ациклического графа, в котором каждый фрагмент требует единичного времени, интервал равен количеству вершин на наидлиннейшем, или критическом, пути (critical path) в ориентированном ациклическом графе.

Фактическое время выполнения многопоточного вычисления зависит не только от его работы и интервала, но и от количества доступных процессоров и от того, как планировщик распределяет фрагменты по процессорам. Обозначая время выполнения многопоточного вычисления на P процессорах, мы будем использовать P в качестве нижнего индекса. Например, время выполнения алгоритма на P процессорах может быть обозначено как T_p . Работа представляет собой время выполнения алгоритма на одном процессоре, т.е. T_1 . Интервал является временем работы при выполнении каждого фрагмента на своем собственном процессоре - другими словами, если у нас имеется неограниченное количество процессоров, - так что интервал мы обозначаем как T_∞ .

Меры производительности

Работа и интервал предоставляют нам нижнюю границу времени выполнения T_p многопоточного вычисления на P процессорах.

За один шаг идеальный параллельный компьютер с P процессорами может выполнить не более P единиц работы; таким образом, за время T_p он может выполнить работу, не превышающую PT_p . Поскольку общая работа составляет T_1 , имеем $PT_p > T_1$. Деление на P дает **правило работы**:

$$T_p > \frac{T_1}{P}. (30.2)$$

Идеальный параллельный компьютер с P процессорами не может работать быстрее машины с неограниченным количеством процессоров. Так что машина с неограниченным количеством процессоров может эмулировать P -процессорную машину, используя только P из своих процессоров. Таким образом, можно сформулировать следующее **правило интервалов**:

$$T_p > T_\infty. (30.3)$$

Определим **ускорение** (speedup) вычислений на P процессорах как отношение T_1/T_p , говорящее о том, во сколько раз вычисления на P процессорах быстрее, чем на 1 процессоре. Согласно правилу работы имеем $T_p > T_1/P$, откуда вытекает, что $T_1/T_p \leq P$. Таким образом, ускорение на P процессорах не может превышать P . Когда ускорение линейно зависит от количества процессоров, т.е. когда $T_1/T_p = \Theta(P)$, вычисления демонстрируют линейное ускорение, а когда $T_1/T_p = P$, мы имеем идеальное линейное ускорение.

Меры производительности

Отношение T_1/T_∞ работы к интервалу дает параллелизм (parallelism) многопоточного вычисления. Параллелизм можно рассматривать с трех точек зрения. Как отношение параллелизм определяет среднее количество работы, которая может быть выполнена параллельно на каждом шаге вдоль критического пути. Как верхняя граница параллелизм дает максимально возможное ускорение, которое может быть достигнуто с помощью произвольного количества процессоров. Наконец, что, вероятно, наиболее важно, параллелизм указывает предел возможности достичь идеального линейного ускорения. В частности, когда количество процессоров превышает уровень параллелизма, вычисления не могут достичь идеального линейного ускорения. Предположим, что $P > T_1/T_\infty$, и в этом случае из правила интервалов вытекает, что ускорение удовлетворяет условию $T_1/T_p < T_1/T_\infty < P$. Кроме того, если число процессоров P в идеальном параллельном компьютере существенно превышает параллелизм - т.е. если $P \gg T_1/T_\infty$, - то $T_1/T_p \ll P$, так что ускорение гораздо меньше, чем количество процессоров. Другими словами, чем больше процессоров мы используем сверх параллелизма, тем менее идеальным становится ускорение.

Определим **зазор параллельности** (parallel slackness) многопоточного вычисления на идеальном параллельном компьютере с P процессорами как отношение $(T_1/T_\infty)/P = T_1/(PT_\infty)$, которое представляет собой множитель, на который параллелизм вычисления превосходит количество процессоров в машине. Таким образом, если зазор меньше 1, мы не можем надеяться достичь идеального линейного ускорения, поскольку $T_1/(PT_\infty) < 1$ и из правила интервалов вытекает, что ускорение на P процессорах удовлетворяет условию $T_1/T_p \leq T_1/T_\infty \leq P$.

Планирование

Хорошая производительность зависит не только от минимизации работы и интервала. Должно также выполняться эффективное распределение фрагментов по процессорам параллельной машины.

Многопоточный планировщик должен выполнять планирование, не зная заранее, когда фрагменты будут запущены или когда они завершатся - он должен работать в **оперативном** (on-line) режиме. Кроме того, хороший планировщик работает распределенно, с учетом баланса загрузки процессоров. Для простоты будем рассматривать **централизованный** (centralized) планировщик, которому в любой момент времени известно глобальное состояние вычисления. В частности, мы проанализируем **жадные планировщики**, назначающие процессорам на каждом временном шаге максимально возможное количество фрагментов. Если на очередном шаге к выполнению готовы как минимум P фрагментов, мы говорим, что этот шаг является **полным**, и жадный планировщик назначает любые P готовых фрагментов процессорам. В противном случае для выполнения готово менее P фрагментов, и мы говорим, что шаг неполон, а планировщик назначает каждый готовый фрагмент своему процессору.

Согласно правилу работы наилучшее время выполнения, на которое мы можем надеяться при наличии P процессоров, составляет $T_p = T_1/P$, а согласно правилу интервала лучшее, на что можно надеяться, - $T_p = T_\infty$.



Планирование

Теорема 30.1

На идеальном параллельном компьютере с P процессорами жадный планировщик выполняет многопоточное вычисление с работой T_1 и интервалом T_∞ за время

$$T_p \leq \frac{T_1}{P} + T_\infty. \quad (30.4)$$

Доказательство. Начнем с рассмотрения полных шагов. На каждом полном шаге P процессоров вместе выполняют общее количество работы, равное P . Предположим для доказательства от противного, что количество полных шагов строго больше, чем $\lceil T_1/P \rceil$. Тогда общее количество работы на полных шагах составляет не менее

$$P(\lceil T_1/P \rceil + 1) = P\lceil T_1/P \rceil + P = T_1 - (T_1 \bmod P) + P > T_1.$$

Таким образом, мы получили противоречие, P процессоров должны выполнить большую работу, чем требует вычисление, и это позволяет заключить, что количество полных шагов не превышает $\lceil T_1/P \rceil$.

Рассмотрим неполный шаг. Пусть G - ориентированный ациклический граф, представляющим все вычисление в целом, и без потери общности предположим, что каждый фрагмент выполняется за единицу времени. Пусть G' представляет собой подграф G , который все еще должен быть выполнен по состоянию на начало неполного шага, и пусть G'' - подграф, который останется невыполненным после этого неполного шага. Найдлиннейший путь в ориентированном ациклическом графе с необходимостью начинается в вершине с нулевой входной степенью. Поскольку неполный шаг жадного планировщика выполняет все фрагменты с нулевой входящей степенью в G' , длина найдлиннейшего пути в G'' должна быть на 1 меньше, чем длина найдлиннейшего пути в G' . Другими словами, неполный шаг уменьшает интервал невыполненного ориентированного ациклического графа на 1. Следовательно, количество неполных шагов не превышает T_∞ .

Планирование

Следствие 30.2

Время выполнения T_p любого многопоточного вычисления при планировании выполнения жадным планировщиком на идеальном параллельном компьютере с P процессорами отличается от оптимального не более чем в 2 раза.

Доказательство. Обозначим через T_p время выполнения, получаемое путем оптимального планирования на машине с P процессорами, и пусть T_1 и T_∞ обозначают соответственно работу и интервал вычисления. Поскольку правила работы и интервала - неравенства (7.2) и (7.3) - дают нам $2 \max(T_1/P, T_\infty)$, из теоремы 7.1 вытекает, что

$$T_p \leq \frac{T_1}{P} + T_\infty \leq 2 \cdot \max\left(\frac{T_1}{P}, T_\infty\right) \leq 2T_p^*.$$

Следствие 30.3

Пусть T_p обозначает время выполнения многопоточного вычисления, полученное жадным планировщиком на идеальном параллельном компьютере с P процессорами, и пусть T_1 и T_∞ представляют собой соответственно работу и интервал вычисления. Тогда, если $P \ll T_1/T_\infty$, имеем $T_p \approx T_1/P$, или, что эквивалентно, ускорение приблизительно равно P .

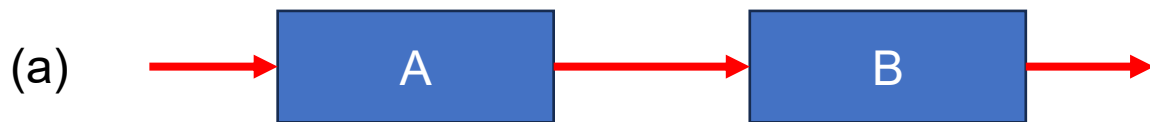
Доказательство. Если мы предположим, что $P \ll T_1/T_\infty$, то мы также имеем $T_\infty \ll T_1/P$, а следовательно, теорема 30.1 дает нам $T_p \leq T_1/P + T_\infty \approx T_1/P$. Поскольку правило работы (30.2) гласит, что $T_p > T_1/P$, мы заключаем, что $T_p \approx T_1/P$, или, что эквивалентно, что ускорение равно $T_1/T_p \sim P$.



Анализ многопоточных алгоритмов

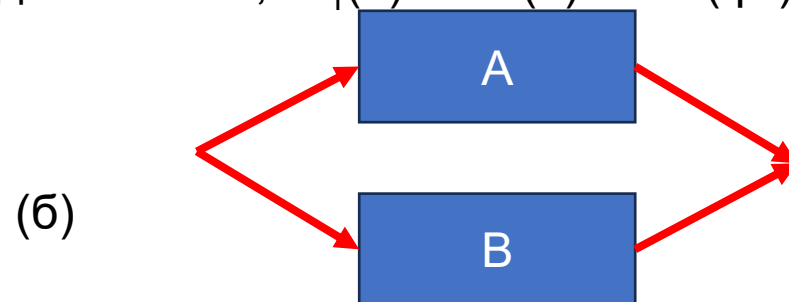
Теперь у нас есть все необходимые инструменты для проведения анализа многопоточных алгоритмов и получения точных границ времен их работы на различном количестве процессоров. Анализ работы достаточно прост и прямолинеен, поскольку ее подсчет - ни что иное, как анализ времени работы обычного последовательного алгоритма. Основные идеи мы изучим на примере программы P-Fib.

Анализ работы $T_1(n)$ алгоритма P-Fib(n) не представляет трудности, поскольку мы уже его проводили. Исходная процедура Fib, по сути, является сериализацией P-Fib, а следовательно, $T_1(n) = T(n) = \Theta(\phi^n)$ согласно уравнению (7.1).



Работа: $T_1(A \cup B) = T_1(A) + T_1(B)$

Интервал: $T_\infty(A \cup B) = T_\infty(A) + T_\infty(B)$



Работа: $T_1(A \cup B) = T_1(A) + T_1(B)$

Интервал: $T_\infty(A \cup B) = \max(T_\infty(A), T_\infty(B))$

а) Когда два подвычисления соединены последовательно, работа их объединения равна сумме отдельных работ, а интервал - сумме отдельных интервалов. (б) Когда два подвычисления соединены параллельно, работа их объединения равна сумме отдельных работ, а интервал - максимальному из отдельных интервалов.

Следовательно, интервал P-Fib(n) можно записать в виде рекуррентного соотношения

$$T_\infty(n) = \max(T_\infty(n-1), T_\infty(n-2)) + \theta(1) = T_\infty(n-1) + \theta(1),$$

решением которого является $T_\infty(n) = \Theta(n)$.

Параллелизм процедуры P-Fib(n) равен $T_1(n)/T_\infty(n) = \Theta(\phi^n/n)$, т.е. быстро растет с ростом n.

Параллельные циклы

Многие алгоритмы включают циклы, все итерации которых могут выполняться параллельно. Такие циклы можно распараллелить с помощью ключевых слов **spawn** и **sync**, но гораздо удобнее явно указать, что итерации в таких циклах могут выполняться одновременно. В нашем псевдокоде такая функциональность обеспечивается ключевым словом **parallel**, предшествующим ключевому слову **for** в соответствующем цикле. Пример - задача умножения матрицы $A = (a_{ij})$ размером $n \times n$ на n -вектор $x = (x_j)$. Полученный в результате n -вектор $y = (y_i)$ определяется как

$$y_i = \sum_{j=1}^n a_{ij} x_j$$

для $i = 1, 2, \dots, n$. Вычислить произведение матрицы на вектор можно путем параллельного вычисления всех элементов y следующим образом.

Mat-Vec(A, x)

1 $n = A.\text{rows}$

2 y - вновь созданный вектор длиной n

3 **parallel for** $i = 1$ to n

4 $y_i = 0$

5 **parallel for** $i = 1$ to n

6 **for** $j = 1$ to n

7 $y_i = y_i + a_{ij} x_j$

8 **return** y



Параллельные циклы

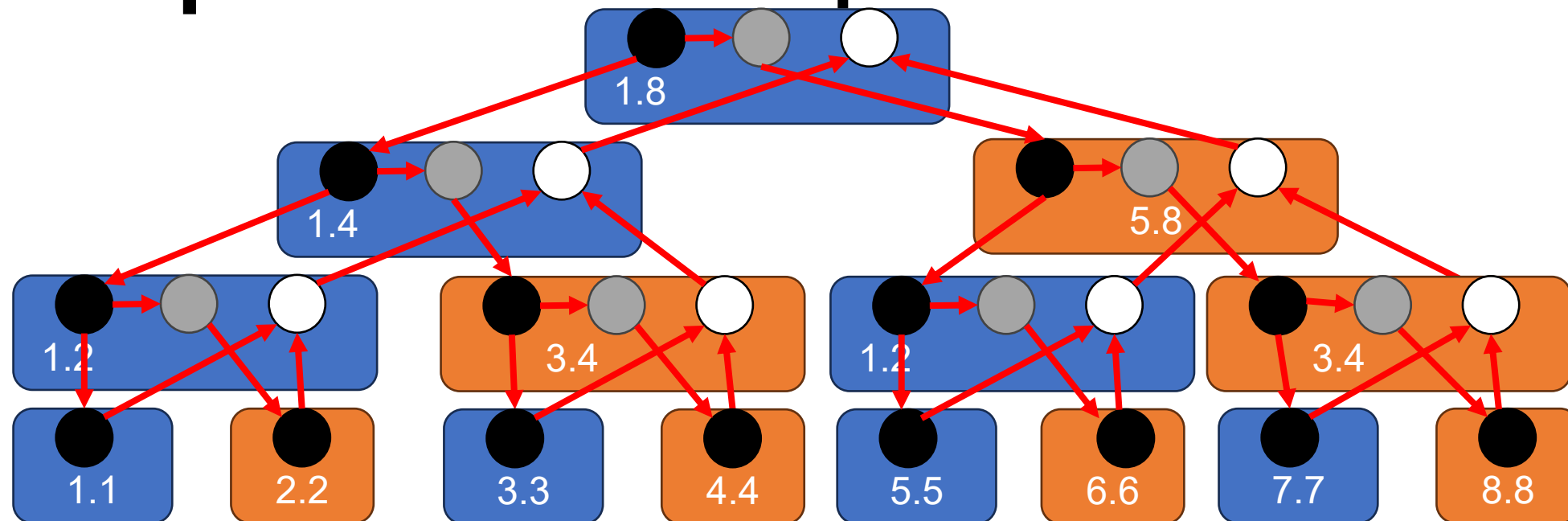
Компилятор может реализовать каждый цикл `parallel for` как подпрограмму с вложенным параллелизмом. Например, цикл `parallel for` в строках 5-7 может быть реализован с помощью вызова `Mat-Vec-Main-Loop(A, x, y, n, 1, n)` в стиле "разделяй и властвуй"; при этом компилятор генерирует вспомогательную подпрограмму `Mat-Vec-Main-Loop` следующим образом.

```
Mat-Vec-Main-Loop(A,x, y,n,i,i')
1 if i == i'
2     for j = 1 to n
3          $y_i = y_i + a_{ij}r_j$ 
4 else mid = [(i + i')/2]
5     spawn Mat-Vec-Main-Loop(A, x, y, n, i, mid)
6     Mat-Vec-Main-Loop(A, x, y, n, mid + 1, i')
7     sync
```

Этот код рекурсивно запускает первую половину итераций цикла параллельно со второй половиной, а затем выполняет команду **sync**, тем самым создавая бинарное дерево выполнения, листья которого представляют отдельные итерации цикла.



Параллельные циклы



Ориентированный ациклический граф, представляющий вычисление `Mat-Vec-Main-Loop(A, x, y, 8, 1, 8)`. Два числа внутри каждого скругленного прямоугольника указывают значения двух последних параметров (i и i' в заголовке процедуры) при запуске или вызове процедуры. Черные круги представляют фрагменты, соответствующие либо базовому случаю, либо части процедуры до запуска `Mat-Vec-Main-Loop` в строке 5; серые круги представляют фрагменты, соответствующие части процедуры, которая вызывает `Mat-Vec-Main-Loop` в строке 6 до ключевого слова `sync` в строке 7, где выполнение приостанавливается до тех пор, пока не будет осуществлен возврат подпрограммы, запущенной в строке 5; белые же круги представляют фрагменты, соответствующие (незначительной) части процедуры после ключевого слова `sync` и до точки возврата из процедуры.

Параллельные циклы

С практической точки зрения динамически многопоточные параллельные платформы иногда **огрубляют** (coarsen) листья рекурсии, выполняя несколько итераций в пределах одного листа, либо автоматически, либо по команде программиста, тем самым снижая накладные расходы рекурсивного запуска. Правда, это снижение достигается ценой уменьшения уровня параллелизма, но, если вычисление имеет достаточный зазор параллельности, близкое к идеальному линейному ускорению при этом в жертву не приносится.

Мы также должны учесть накладные расходы на рекурсивные запуски в ходе анализа интервала конструкции параллельного цикла. Поскольку глубина рекурсивных вызовов логарифмически зависит от количества итераций, в случае параллельного цикла с p итерациями, в котором i -я итерация имеет интервал $itera(i)$, интервал цикла равен

$$T_{\infty}(n) = \Theta(\lg n) + \max_{1 \leq i \leq n} itera_{\infty}(i) .$$

Например, для процедуры Mat-VEC в случае матрицы размером $n \times n$ параллельный цикл инициализации в строках 3 и 4 имеет интервал $\Theta(\lg n)$, поскольку рекурсивный запуск доминирует над константной работой в каждой итерации. Интервал дважды вложенных циклов в строках 5-7 составляет $\Theta(n)$, поскольку каждая итерация внешнего цикла **parallel for** содержит n итераций внутреннего (последовательного) цикла **for**. Поскольку работа равна $\Theta(n^2)$, уровень параллелизма имеет значение $\Theta(n^2)/\Theta(n) = \Theta(n)$.



УСЛОВИЯ ГОНКИ

Многопоточный алгоритм является **детерминированным** (deterministic), если он всегда приводит к одним и тем же результатам при одних и тех же входных данных, независимо от того, как именно планируются команды в многоядерном компьютере. Алгоритм является **недетерминированным**, если его поведение может меняться от выполнения к выполнению. Зачастую многопоточные алгоритмы, которые должны быть детерминированными, таковыми не являются, поскольку содержат "гонку детерминированности" (determinacy race).

Гонка детерминированности осуществляется в том случае, когда две логически параллельные команды обращаются к одной и той же ячейке памяти и как минимум одна из них выполняет запись. Приведенная далее процедура иллюстрирует условие гонки.

Mat-Vec-Wrong(A,x)

```
1 n = A.rows
2 y - вновь созданный вектор длиной n
3 parallel for i = 1 to n
4      $y_i = 0$ 
5 parallel for i = 1 to n
6     parallel for j = 1 to n
7          $y_i = y_i + a_{ij}x_j$ 
8 return y
```

К сожалению, эта процедура некорректна из-за гонки при обновлении y_i в строке 7, выполняемой параллельно для всех n значений j . Дз - предлагается разработать корректную реализацию алгоритма с интервалом $\Theta(n^2)$.

Многопоточный алгоритм "разделяй и властвуй" для умножения матриц

Матрицы размером $n \times n$ можно перемножить последовательно за время $\Theta(n^{\lg 7}) = O(n^{2.81})$, воспользовавшись стратегией "разделяй и властвуй" Штрассена. Естественным образом возникает вопрос о многопоточной версии этого алгоритма. Начнем с многопоточной версии более простого алгоритма "разделяй и властвуй".

Вспомним, что процедура Square-Matrix-Multiply-Recursive, которая умножает две матрицы, A и B , размером $n \times n$ и дает в результате матрицу C того же размера, основывается на разбиении каждой из трех матриц на четыре подматрицы размером $n/2 \times n/2$:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

После этого мы можем переписать умножение матриц следующим образом:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} A_{11}B_{11} & A_{11}B_{12} \\ A_{21}B_{11} & A_{21}B_{12} \end{pmatrix} + \begin{pmatrix} A_{12}B_{21} & A_{12}B_{22} \\ A_{22}B_{21} & A_{22}B_{22} \end{pmatrix}$$

Итак, для умножения двух матриц размером $n \times n$ мы выполняем восемь умножений матриц размером $n/2 \times n/2$ и одно сложение матриц размером $n \times n$. Приведенный ниже псевдокод реализует эту стратегию "разделяй и властвуй" с использованием вложенного параллелизма. В отличие от процедуры Square-Matrix-Multiply-Recursive, на которой она основана, процедура P-Matrix-Multiply-Recursive получает выходную матрицу в качестве параметра, чтобы избежать излишнего выделения памяти для матриц.

Многопоточный алгоритм "разделяй и властвуй" для умножения матриц

P-Matrix-Multiply-Recursive(C, A, B)

1 $n = A.rows$

2 if $n == 1$

3 $c_{11} = a_{11}b_{11}$

4 else пусть T представляет собой новую матрицу размером $n \times n$

5 Разбиение A, B, C и T на подматрицы размером $n/2 \times n/2$ $A_{11}, A_{12}, A_{21}, A_{22}; B_{11}, B_{12}, B_{21}, B_{22}; C_{11}, C_{12}, C_{21}, C_{22};$ и $T_{11}, T_{12}, T_{21}, T_{22};$ соответственно

6 spawn P-Matrix-Multiply-Recursive(C_{11}, A_{11}, B_{11})

7 spawn P-Matrix-Multiply-Recursive(C_{12}, A_{11}, B_{12})

8 spawn P-Matrix-Multiply-Recursive(C_{21}, A_{21}, B_{11})

9 spawn P-Matrix-Multiply-Recursive(C_{22}, A_{21}, B_{12})

10 spawn P-Matrix-Multiply-Recursive(T_{11}, A_{12}, B_{21})

11 spawn P-Matrix-Multiply-Recursive(T_{12}, A_{12}, B_{22})

12 spawn P-Matrix-Multiply-Recursive(T_{21}, A_{22}, B_{21})

13 P-Matrix-Multiply-Recursive(T_{22}, A_{22}, B_{22})

14 sync

15 parallel for $i = 1$ to n

16 parallel for $j = 1$ to n

17 $c_{ij} = c_{ij} + t_{ij}$

Многопоточный алгоритм "разделяй и властвуй" для умножения матриц

В рекурсивном случае разбиение выполняется за время $\Theta(1)$, затем выполняется восемь рекурсивных умножений матриц размером $n/2 \times n/2$, и наконец выполняется работа $\Theta(n^2)$ по сложению двух матриц размером $n \times n$. Таким образом, рекуррентное соотношение для работы $M_1(n)$ имеет вид

$$M_1(n) = 8M_1(n/2) + \Theta(n^2) = \Theta(n^3)$$

согласно случаю 1 основной теоремы. Для определения интервала $M_\infty(n)$ процедуры P-Matrix-Multiply-Recursive сначала заметим, что интервал для разбиения равен $\Theta(1)$ и над ним доминирует интервал $\Theta(\lg n)$ дважды вложенных циклов parallel for в строках 15-17. Поскольку восемь параллельных рекурсивных вызовов работают с матрицами одинакового размера, максимальным интервалом каждого рекурсивного вызова является интервал любого из них. Следовательно, рекуррентное соотношение для интервала $M_\infty(n)$ процедуры P-Matrix-Multiply-Recursive имеет вид

$$M_\infty(n) = M_\infty\left(\frac{n}{2}\right) + \Theta(\lg n). \quad (30.7)$$

Это рекуррентное соотношение не подпадает ни под один из случаев основной теоремы. Рекуррентного соотношения (7.7) является $M_\infty(n) = \Theta(\lg^2 n)$. Теперь, когда мы знаем работу и интервал процедуры P-Matrix-Multiply-Recursive, можно вычислить уровень ее параллелизма как $M_1(n)/M_\infty(n) = \Theta(n^3/\lg^2 n)$ и увидеть, что он весьма высок.

Многопоточный метод Штрассена

1. Разбиваем входные матрицы A и B и выходную матрицу C на подматрицы размером $n/2 \times n/2$, как в (7.6). Работа и интервал этого шага составляют $\Theta(1)$ при использовании пересчета индексов.

2. Создаем 10 матриц $S_1, S_2, \dots, S_{10}$, каждая из которых имеет размер $n/2 \times n/2$ и представляет собой сумму или разность двух матриц, созданных в п. 1. Все 10 матриц создаются ценой работы $\Theta(n^2)$ и интервала $\Theta(\lg n)$ с помощью дважды вложенных циклов **parallel for**.

3. Используя созданные в п. 1 подматрицы и 10 матриц, созданных в п. 2, рекурсивно запускаем вычисление семи произведений матриц $P_1, P_2, \dots, P_7$ размером $n/2 \times n/2$.

4. Вычисляем искомые подматрицы $C_{11}, C_{12}, C_{21}, C_{22}$ результирующей матрицы C путем сложения и вычитания различных комбинаций матриц P_i , вновь используя дважды вложенные циклы **parallel for**. Вычисление всех четырех подматриц имеет работу $\Theta(n^2)$ и интервал $\Theta(\lg n)$.

Для анализа этого алгоритма сначала заметим, что, поскольку сериализация совпадает с исходным последовательным алгоритмом, работа нашего алгоритма равна времени работы сериализации, а именно - $\Theta(n^{\lg 7})$. Как и в случае процедуры P-Matrix-Multiply-Recursive для интервала можно записать рекуррентное соотношение. В данном случае параллельно выполняются семь рекурсивных вызовов, но поскольку все они работают с матрицами одного и того же размера, мы получаем такое же рекуррентное соотношение (30.7), как и в случае процедуры P-Matrix-Multiply-Recursive, решением которого является $\Theta(\lg^2 n)$.

Таким образом, уровень параллелизма многопоточного алгоритма Штрассена равен $\Theta(n^{\lg 7} / \lg^2 n)$. Это высокий параллелизм, хотя и меньший, чем в случае процедуры P-Matrix-Multiply-Recursive.